МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ТУЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт прикладной математики и компьютерных наук

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «ПРОГРАММИРОВАНИЕ»

_____

(тема курсовой работы)

Студент гр. _____ _____ _____

(индекс группы) (подпись и дата) (инициалы и фамилия)

Руководитель _____ _____ _____

(должность и ученая степень) (подпись и дата) (инициалы и фамилия)

ТУЛА 2026

ЗАДАНИЕ

на курсовую работу по дисциплине

«Программирование»

студент(ка) Фураев Владислав Викторович гр.221341

ТЕМА: Прикладной программный продукт «Прикладной программный продукт «Ресторанный бизнес / общепит. Обработка заказов: зал / кухня». ».

Исходные данные согласно варианту №30

Задание получил: _____Фураев В.В.

Дата выдачи задания : 11.09.2025

Задание выдал: _____Креслинь М.В.

Срок защиты курсовой работы: _____

Замечания консультанта: _____

_____

К защите допущен. Консультант работы _____

"___"_____20__г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 7
1.1 Проблематика и актуальность 7
1.2 Анализ существующих решений 8
1.3 Цель и задачи проекта 8
2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ 9
2.1 Назначение разработки 9
2.2 Функциональные требования 9
2.2.1 Модуль зала обслуживания 9
2.2.2 Модуль кухни 9
2.2.3 Общие функции 10
2.3 Требования к интерфейсу пользователя 10
2.3.1 Общие требования 10
2.3.2 Интерфейс зала 10
2.3.3 Интерфейс кухни 11
2.4 Стадии и этапы разработки 11
3 МОДЕЛИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММЫ НА ОСНОВЕ ПРИНЦИПОВ ООП 11
3.1 Архитектура программного обеспечения 11
3.2 Проектирование классов 12
3.2.1 Класс MenuItem 12
3.2.2 Класс Table 12
3.2.3 Класс OrderItem 13
3.2.4 Класс Order 13
3.2.5 Класс KitchenManager 14
3.2.6 Класс DataManager 14
3.3 Диаграмма классов 15
4 ОПИСАНИЕ ПРОГРАММЫ 17
4.1 Функциональное назначение 17
4.2 Описание логической структуры 17

4.2.1 Модульная структура 17
4.2.2 Поток данных 18
4.2.3 Алгоритмы и логика 18
4.3 Входные данные 19
4.3.1 Типы входных данных 19
4.3.2 Источники данных 19
4.4 Выходные данные 20
4.4.1 Визуальная информация 20
4.4.2 Бизнес-информация 20
4.5 Используемые технологии и библиотеки 20
4.5.1 Язык программирования и стандарт 20
4.5.2 Платформенные API 20
4.5.3 Прагмы и настройки компилятора 21
5 ИНСТРУКЦИЯ ПО ЭКСПЛУАТАЦИИ ПРОГРАММЫ 21
5.1 Назначение программы 21
5.2 Системные требования 21
5.2.1 Минимальные требования 21
5.2.2 Рекомендуемые требования 22
5.2.3 Зависимости 22
5.3 Описание интерфейса пользователя 22
5.3.1 Режим зала обслуживания 22
5.3.2 Режим кухни 23
5.4 Работа с программой 24
5.4.1 Создание нового заказа 24
5.4.2 Добавление блюд в заказ 25
5.4.3 Передача заказа на кухню 26
5.4.4 Работа на кухне 27
5.4.5 Завершение заказа 28
5.4.6 Мониторинг статистики 29
5.5 Описание контрольного примера 30
5.5.1 Тестовый сценарий 30
5.5.2 Проверка результатов 31

6 ТЕСТИРОВАНИЕ И ОТЛАДКА ПРОГРАММЫ 31
6.1 Методология тестирования 31
6.1.1 Виды тестирования 31
6.1.2 Инструменты тестирования 32
6.2 Тестовые сценарии 32
6.2.1 Сценарий 1: Создание и обработка заказа 32
6.2.2 Сценарий 2: Обработка нескольких заказов 32
6.2.3 Сценарий 3: Обработка ошибок 32
6.3 Результаты тестирования 33
6.3.1 Функциональное тестирование 33
6.3.2 Нефункциональное тестирование 33
6.3.3 Найденные и исправленные ошибки 33
7 ЗАКЛЮЧЕНИЕ 34
7.1 Достигнутые результаты 34
7.2 Соответствие требованиям 34
7.3 Научно-практическая значимость 34
7.4 Перспективы развития 35
7.5 Выводы 35
8 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 36
8.1 Нормативные документы 36
8.2 Книги и учебные пособия 36
8.3 Статьи и научные публикации 36
8.4 Интернет-ресурсы 36
8.5 Программное обеспечение 36
9 ПРИЛОЖЕНИЯ 37
Приложение А. Листинг программы 37
Приложение Б. Диаграммы UML 60
Б.1. Диаграмма последовательности 60
Б.2. Диаграмма состояний заказа 61
Приложение В. Руководство по установке 62

ВВЕДЕНИЕ

В современных условиях развития ресторанного бизнеса и сферы общественного питания особую актуальность приобретают вопросы автоматизации ключевых бизнес-процессов. Одной из наиболее важных задач является оптимизация обработки заказов, которая напрямую влияет на качество обслуживания клиентов, операционную эффективность заведения и конечную прибыльность предприятия.

Традиционные методы организации работы в ресторанах, основанные на бумажных носителях и устных коммуникациях между персоналом зала и кухни, демонстрируют ряд существенных недостатков. К ним относятся: длительные задержки в передаче информации, высокая вероятность ошибок при интерпретации заказов, сложности в отслеживании статусов приготовления блюд, неэффективное распределение ресурсов кухни и отсутствие оперативной аналитической информации для принятия управленческих решений.

Развитие информационных технологий и объектно-ориентированного программирования открывает новые возможности для создания специализированных программных решений, способных кардинально улучшить процессы управления заведениями общественного питания. Особое значение в этом контексте приобретает разработка систем, обеспечивающих сквозную автоматизацию обработки заказов – от момента принятия заказа от клиента до его финального завершения и оплаты.

Данная курсовая работа посвящена проектированию и разработке прикладного программного продукта «Ресторанный бизнес / общепит. Обработка заказов: зал / кухня». Основная цель работы заключается в создании эффективного инструмента автоматизации, построенного на принципах объектно-ориентированного программирования и предназначенного для координации взаимодействия между залом обслуживания и производственной зоной кухни.

Актуальность разработки обусловлена следующими факторами:

1. **Повышение качества обслуживания** – минимизация времени обслуживания и сокращение количества ошибок
2. **Оптимизация внутренних процессов** – эффективное распределение задач между персоналом
3. **Улучшение управляемости** – получение оперативной информации о состоянии бизнес-процессов
4. **Рост прибыльности** – снижение операционных издержек и увеличение оборачиваемости столов

В рамках выполнения работы будут решены следующие задачи:

- Проведен анализ предметной области и существующих решений
- Разработана объектно-ориентированная архитектура системы
- Реализованы основные классы и их взаимодействия
- Создан графический пользовательский интерфейс на основе Windows API
- Протестирована работоспособность программного продукта
- Подготовлена полная документация в соответствии с требованиями ГОСТ

Особое внимание в работе уделяется применению современных методологий объектно-ориентированного проектирования, принципам SOLID, паттернам проектирования и лучшим практикам разработки программного обеспечения на языке C++. Разрабатываемая система демонстрирует практическое применение теоретических знаний, полученных в ходе изучения дисциплины «Программирование», и служит примером создания полноценного прикладного решения для реальной предметной области.

Результатом работы станет готовый к использованию программный продукт, способный повысить эффективность работы ресторанов и заведений общественного питания за счет автоматизации ключевых бизнес-процессов и обеспечения точной координации между различными функциональными зонами предприятия.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Проблематика и актуальность

В современном ресторанном бизнесе эффективная обработка заказов является критически важным процессом, влияющим на качество обслуживания клиентов и операционную эффективность заведения. Традиционные методы обработки заказов, основанные на бумажных бланках и устных коммуникациях между залом и кухней, имеют ряд существенных недостатков:

1. **Задержки в передаче информации** – время между принятием заказа от клиента и его поступлением на кухню может составлять несколько минут.
2. **Ошибки при передаче данных** – неправильно понятые или записанные заказы приводят к недовольству клиентов.
3. **Отсутствие реального отслеживания статусов** – официанты не знают точного времени приготовления блюд, а повара – когда подавать готовые заказы.

4. **Сложность учета и отчетности** – ручной подсчет выручки, статистики по блюдам и анализа эффективности работы персонала.
5. **Неэффективное использование ресурсов** – простаивающие повара при отсутствии заказов и перегруженность в пиковые часы.

В условиях высокой конкуренции в сфере общественного питания автоматизация процессов обработки заказов становится необходимостью для повышения качества обслуживания и оптимизации внутренних бизнес-процессов.

1.2 Анализ существующих решений

На рынке представлены различные системы автоматизации ресторанного бизнеса:

1. **Полнофункциональные POS-системы**:

- iiko
- R-Keeper
- Poster
- *Преимущества*: комплексный функционал, интеграция с фискальными регистраторами.
- *Недостатки*: высокая стоимость, сложность настройки, избыточность для небольших заведений.

2. **Облачные решения**:

- Yandex.EDA для ресторанов
- SimpleOne
- *Преимущества*: доступность, простота использования.
- *Недостатки*: зависимость от интернета, ограниченная кастомизация.

3. **Самописные решения**:

- Разрабатываются под конкретные нужды заведения.
- *Преимущества*: полное соответствие требованиям.
- *Недостатки*: высокая стоимость разработки, требуются технические специалисты.

1.3 Цель и задачи проекта

Цель проекта: разработка специализированного программного обеспечения для автоматизации обработки заказов в ресторане с разделением на модули зала и кухни.

Задачи проекта:

1. Анализ бизнес-процессов обработки заказов в ресторане.
2. Проектирование объектно-ориентированной архитектуры системы.

3. Разработка модуля управления столами и меню.
4. Реализация системы обработки заказов с отслеживанием статусов.
5. Создание модуля управления кухней с автоматической логикой приготовления.
6. Разработка системы финансового учета и отчетности.
7. Создание интуитивно понятного графического интерфейса на основе Windows API.
8. Тестирование и отладка программного продукта.

2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1 Назначение разработки

Разработать прикладной программный продукт для автоматизации обработки заказов в ресторанах и заведениях общественного питания с разделением на две основные зоны: зал обслуживания и кухня.

Область применения: рестораны, кафе, столовые, бары, пиццерии.

Классификация по ГОСТ 34.003-90: информационно-управляющая система.

2.2 Функциональные требования

2.2.1 Модуль зала обслуживания

1. **Управление столами**:
   - Отображение статусов столов (свободен/занят)
   - Информация о вместимости столов
   - Привязка заказов к конкретным столам
2. **Работа с меню**:
   - Категоризация блюд (основные блюда, салаты, напитки, десерты)
   - Отображение цен и времени приготовления
   - Кухонные заметки для поваров
3. **Обработка заказов**:
   - Создание новых заказов
   - Добавление/удаление позиций из заказа
   - Просмотр деталей заказа
   - Передача заказа на кухню
   - Завершение заказа после оплаты

2.2.2 Модуль кухни

1. **Управление заказами**:

- Прием заказов из зала
- Отображение статусов заказов
- Автоматическое распределение блюд по очередности

2. **Контроль приготовления**:

- Отметка о начале приготовления каждого блюда
- Фиксация времени готовности
- Добавление поварских заметок
- Отметка о подаче блюд

3. **Мониторинг и отчетность**:

- Лог событий кухни
- Статистика по выполненным заказам
- Учет выручки за день

2.2.3 Общие функции

1. **Синхронизация данных** – автоматическое обновление статусов между залом и кухней
2. **Финансовый учет** – подсчет выручки, средней суммы чека
3. **Валидация данных** – проверка корректности вводимой информации
4. **Обработка ошибок** – защита от некорректных действий пользователя

2.3 Требования к интерфейсу пользователя

2.3.1 Общие требования

1. **Разделение на режимы** – четкое разграничение интерфейсов для зала и кухни
2. **Интуитивность** – понятная навигация, минимальное обучение персонала
3. **Быстродействие** – мгновенный отклик на действия пользователя
4. **Наглядность** – цветовое кодирование статусов, понятные иконки

2.3.2 Интерфейс зала

1. **Главное окно зала**:

- Панель управления столами
- Список меню с категориями
- Панель активных заказов
- Детализация выбранного заказа
- Кнопки основных действий

2. **Визуализация статусов**:

- Свободные столы – зеленый цвет
- Занятые столы – красный цвет
- Заказы в процессе – желтый цвет
- Готовые к завершению – синий цвет

2.3.3 Интерфейс кухни

1. **Главное окно кухни**:

- Список активных заказов с детализацией
- Лог событий кухни
- Статистическая панель
- Кнопки управления приготовлением

2. **Детализация заказов**:

- Количество блюд в ожидании/приготовлении/готовности
- Время принятия заказа
- Текущее готовящееся блюдо
- Заметки официантов и поваров

2.4 Стадии и этапы разработки

В соответствии с ГОСТ 34.601-90:

1. **Техническое задание** – анализ требований, определение функционала
2. **Эскизный проект** – проектирование архитектуры, интерфейсов, форматов данных
3. **Технический проект** – детальное проектирование классов, алгоритмов, структур данных
4. **Рабочий проект** – написание кода, создание интерфейса, интеграция модулей
5. **Внедрение** – тестирование, документация, демонстрация работы

3 МОДЕЛИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММЫ НА ОСНОВЕ ПРИНЦИПОВ ООП

3.1 Архитектура программного обеспечения

Система построена по модульной архитектуре с четким разделением ответственности:

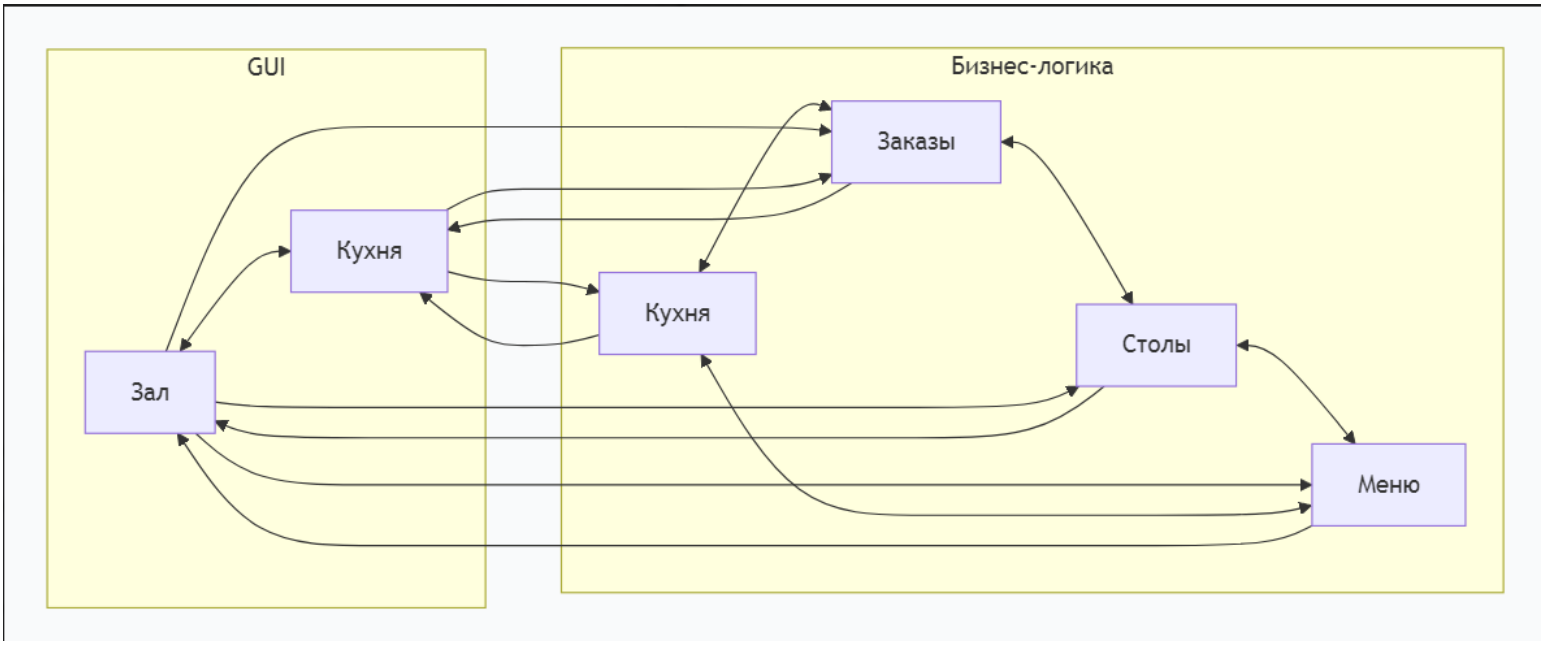


Рисунок 1 - Архитектура программного обеспечения

Слои архитектуры:

1. **Слой представления** – графический интерфейс на Windows API
2. **Слой бизнес-логики** – классы управления данными и процессами
3. **Слой данных** – хранение информации в оперативной памяти

3.2 Проектирование классов

3.2.1 Класс MenuItem

Назначение: представление блюда в меню.

Атрибуты:

- id – уникальный идентификатор блюда
- name – название блюда
- category – категория (основные блюда, салаты и т.д.)
- price – цена
- prepTime – время приготовления в минутах
- kitchenNotes – заметки для поваров

Принципы ООП:

- **Инкапсуляция** – данные защищены, доступ через методы
- **Полиморфизм** – может использоваться в разных контекстах

3.2.2 Класс Table

Назначение: представление стола в зале.

Атрибуты:

- number – номер стола
- capacity – вместимость (количество персон)
- isOccupied – статус занятости
- location – расположение (зал, терраса, VIP)

Методы:

- Конструкторы для инициализации
- Методы изменения статуса

3.2.3 Класс OrderItem

Назначение: элемент заказа (конкретное блюдо с статусом приготовления).

Атрибуты:

- item – ссылка на блюдо из меню
- status – статус ("В ожидании", "Готовится", "Готово", "Подано")
- startTime, finishTime – время начала и окончания приготовления
- chefNotes – заметки повара

Особенности:

- Связь с MenuItem через композицию
- Отслеживание временных меток

3.2.4 Класс Order

Назначение: представление заказа клиента.

Атрибуты:

- id – уникальный идентификатор заказа
- tableNumber – номер стола
- items – список элементов заказа
- status – общий статус заказа
- createdTime, kitchenAcceptTime – временные метки
- waiterNotes – заметки официанта
- currentCookingIndex – индекс текущего готовящегося блюда

Методы:

- getTotal() – расчет общей суммы
- addItem() – добавление блюда
- isReady(), isServed() – проверки статусов

- getNextItemToCook(), getNextItemToServe() – логика очереди

Принципы ООП:

- **Агрегация** – содержит коллекцию OrderItem
- **Полиморфизм** – разные состояния заказа

3.2.5 Класс KitchenManager

Назначение: управление процессами на кухне.

Атрибуты:

- activeOrders – активные заказы на кухне
- kitchenLog – лог событий
- currentChef – текущий повар
- ordersCompletedToday, dailyRevenue – статистика

Методы:

- acceptOrder() – прием заказа с зала
- startCookingItem(), completeItem() – управление приготовлением
- markItemServed() – отметка о подаче
- startNextItem(), serveNextItem() – автоматическая логика
- getKitchenStatistics() – формирование статистики

Паттерны проектирования:

- **Фасад** – единая точка управления кухней
- **Наблюдатель** – отслеживание изменений статусов

3.2.6 Класс DataManager

Назначение: центральный менеджер данных системы.

Атрибуты:

- tables, menu, orders – коллекции данных
- nextOrderId – счетчик для генерации ID
- kitchen – экземпляр KitchenManager

Методы:

- initializeData() – инициализация тестовых данных
- CRUD-операции для всех сущностей
- createOrder(), addItemToOrder() – бизнес-логика
- sendOrderToKitchen() – интеграция с кухней
- getTotalRevenue() – финансовый учет

Принципы ООП:

- **Композиция** – содержит KitchenManager
- **Единственная ответственность** – управление данными

3.3 Диаграмма классов

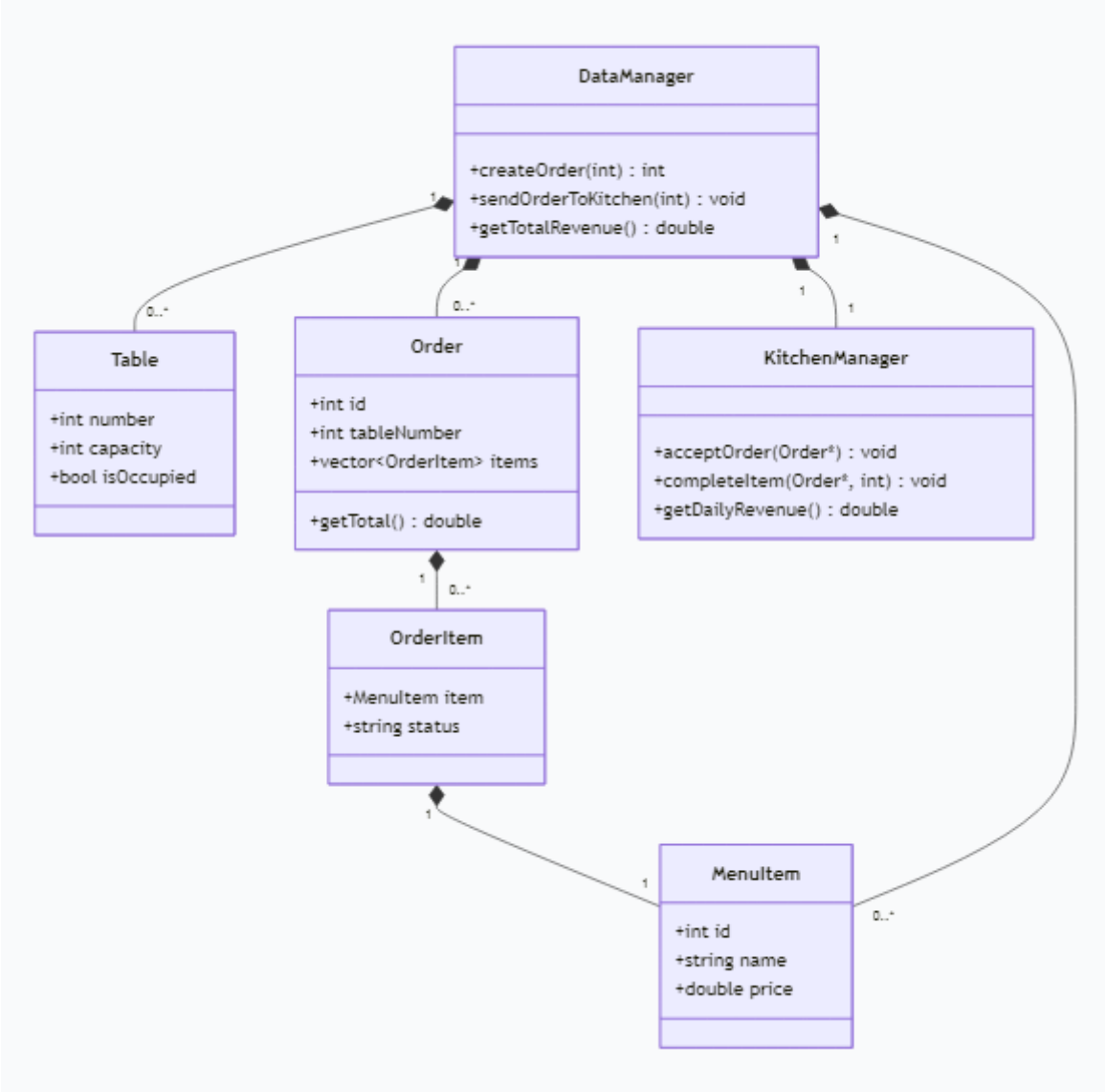


Рисунок 2 – **Связь классов**

Типы связей:

1. **Композиция (сплошной ромб)** *--

Значение: Сильная связь "часть-целое". Объект-часть не может существовать без объекта-целого. При удалении целого удаляются все его части.

Примеры в программе:

- Order *-- OrderItem – Заказ СОСТАВЛЕН из элементов заказа. Без заказа элементы заказа не имеют смысла.
- DataManager *-- KitchenManager – Менеджер данных СОДЕРЖИТ менеджер кухни. Кухня не существует отдельно от системы.
- OrderItem *-- MenuItem – Элемент заказа СОДЕРЖИТ информацию о блюде из меню.

2. **Агрегация (пустой ромб)** o--

Значение: Слабая связь "часть-целое". Объект-часть может существовать независимо от объекта-целого.

Примеры в программе:

- DataManager o-- Table – Система ИСПОЛЬЗУЕТ столы, но столы могут существовать отдельно.
- DataManager o-- MenuItem – Система РАБОТАЕТ с блюдами меню, но меню может быть отдельным.
- DataManager o-- Order – Система УПРАВЛЯЕТ заказами, но заказы могут существовать в других системах.

3. **Зависимость (пунктирная стрелка)** ..>

Значение: Один класс использует другой временно. Изменение одного класса может повлиять на другой.

Примеры в программе:

- KitchenManager ..> Order* – Кухня РАБОТАЕТ с заказами через указатели, но не владеет ими.

Кратности (цифры в кавычках):

- "1" – один объект
- "0..*" – от нуля до многих объектов
- "1..*" – от одного до многих объектов

Примеры:

- Order "1" *-- "0..*" OrderItem – Один заказ МОЖЕТ СОДЕРЖАТЬ от 0 до многих элементов заказа.
- DataManager "1" *-- "1" KitchenManager – Один DataManager ТОЧНО СОДЕРЖИТ один KitchenManager.

Архитектурный смысл:

1. **DataManager** – центральный класс, управляющий всей системой
2. **Order** – бизнес-сущность "заказ", содержащая детали
3. **KitchenManager** – специализированный модуль для кухни
4. **Table/MenuItem** – вспомогательные сущности данных

Принцип: Strong Composition (композиция) используется для логически связанных объектов, Weak Aggregation (агрегация) – для независимых данных.

4 ОПИСАНИЕ ПРОГРАММЫ

4.1 Функциональное назначение

Программа предназначена для автоматизации обработки заказов в ресторанах и заведениях общественного питания. Она обеспечивает:

1. **Эффективное управление столами** – отслеживание занятости, вместимости
2. **Удобную работу с меню** – категоризация, цены, время приготовления
3. **Сквозную обработку заказов** – от создания до завершения и оплаты
4. **Координацию между залом и кухней** – мгновенная передача информации
5. **Автоматизацию процессов приготовления** – умная очередь блюд
6. **Финансовый учет** – подсчет выручки, статистика
7. **Мониторинг производительности** – анализ работы кухни и персонала

4.2 Описание логической структуры

4.2.1 Модульная структура

Программа делится на два основных режима:

1. Режим зала (Hall Mode)

2. Режим кухни (Kitchen Mode)

Основные компоненты:

1. DataManager – центральный контроллер данных

2. KitchenManager – управление кухонными процессами

3. GUI Manager – управление интерфейсом Windows

4. Business Logic – правила обработки заказов

4.2.2 Поток данных

1. Официант создает заказ → DataManager.createOrder()

2. Добавляет блюда → DataManager.addItemToOrder()

3. Передает на кухню → DataManager.sendOrderToKitchen()

↓

4. Кухня принимает заказ → KitchenManager.acceptOrder()

5. Начинает приготовление → KitchenManager.startNextItem()

6. Завершает блюдо → KitchenManager.completeItem()

7. Отмечает подачу → KitchenManager.markItemServed()

↓

8. Официант завершает заказ → DataManager.completeOrder()

4.2.3 Алгоритмы и логика

1. **Алгоритм очереди блюд**:

```
int Order::getNextItemToCook() {
    for (int i = 0; i < items.size(); i++) {
        if (items[i].status == "В ожидании") {
            return i; // Нашли блюдо для приготовления
        }
    }
    return -1; // Нет блюд для приготовления
}
```

2. **Логика автоматического старта**:

```
        void KitchenManager::completeItem(Order* order, int itemIndex) {
            // Завершаем текущее блюдо
            order->items[itemIndex].status = "Готово";

            // Автоматически начинаем следующее
            if (startNextItem(order)) {
                kitchenLog.push_back("Автоматически начато следующее
        блюдо");
            }
}
```

3. **Валидация завершения заказа**:

```
    bool Order::canBeCompleted() const {
        return isServed() && status != "Завершен";
    }
```

4.3 Входные данные

4.3.1 Типы входных данных

1. **Данные о столиках** (инициализируются программно):
   - Номер стола
   - Вместимость
   - Расположение
   - Статус занятости
2. **Данные меню** (инициализируются программно):
   - Названия блюд
   - Категории
   - Цены
   - Время приготовления
   - Кухонные заметки
3. **Действия пользователя**:
   - Выбор стола для создания заказа
   - Выбор блюд из меню
   - Команды управления (создать, передать, завершить)
   - Переключение между режимами зала и кухни

4.3.2 Источники данных

1. **Программная инициализация** – тестовые данные в коде

2. **Пользовательский ввод** – через графический интерфейс
3. **Системное время** – для отметки времени событий

4.4 Выходные данные

4.4.1 Визуальная информация

1. **Интерфейс зала**:
   - Список столов со статусами
   - Меню с ценами
   - Активные заказы с детализацией
   - Общая выручка за день
2. **Интерфейс кухни**:
   - Активные заказы с прогрессом приготовления
   - Лог событий кухни
   - Статистика работы
   - Информация о текущем поваре

4.4.2 Бизнес-информация

1. **Финансовые данные**:
   - Выручка за день
   - Суммы по заказам
   - Средний чек
2. **Операционная статистика**:
   - Количество завершенных заказов
   - Время обработки заказов
   - Загрузка кухни
   - Эффективность работы персонала
3. **Журналы событий**:
   - Лог кухни (принятие, начало, завершение, подача)
   - История изменений статусов

4.5 Используемые технологии и библиотеки

4.5.1 Язык программирования и стандарт

- **Язык**: C++17
- **Компилятор: GCC 14.2 (MinGW-w64), среда разработки Code::Blocks 25.03**
- **Стандартная библиотека**: STL (Standard Template Library)

4.5.2 Платформенные API

1. **Windows API**:

- User32.dll – создание окон и элементов управления
- Comctl32.lib – библиотека общих элементов управления
- Основные функции: CreateWindow(), SendMessage(), MessageBox()

2. **Используемые заголовочные файлы**:

```
#include <windows.h>      // Основные функции Windows API
#include <commctrl.h>     // Общие элементы управления
#include <vector>         // Контейнер vector
#include <string>         // Строки C++
#include <sstream>        // Строковые потоки
#include <algorithm>      // Алгоритмы STL
#include <map>            // Ассоциативный массив
#include <ctime>          // Работа со временем
```

4.5.3 Прагмы и настройки компилятора

```
#pragma comment(linker, "/subsystem:windows /entry:mainCRTStartup")
// Указывает, что программа использует оконный интерфейс

#pragma comment(lib, "comctl32.lib")
// Подключает библиотеку общих элементов управления
```

5 ИНСТРУКЦИЯ ПО ЭКСПЛУАТАЦИИ ПРОГРАММЫ

5.1 Назначение программы

Программа «Ресторан: Управление заказами» предназначена для:

- Автоматизации процессов приема и обработки заказов в ресторанах
- Координации работы между залом обслуживания и кухней
- Учета занятости столов и управления меню
- Отслеживания статусов приготовления блюд
- Подсчета выручки и анализа эффективности работы

5.2 Системные требования

5.2.1 Минимальные требования

- **Операционная система**: Windows 7 или выше
- **Процессор**: 1 GHz или выше
- **Оперативная память**: 512 MB
- **Свободное место на диске**: 10 MB
- **Разрешение экрана**: 1024×768

5.2.2 Рекомендуемые требования

- **Операционная система**: Windows 10/11
- **Процессор**: 2 GHz, 2 ядра
- **Оперативная память**: 1 GB
- **Свободное место на диске**: 20 MB
- **Разрешение экрана**: 1920×1080

5.2.3 Зависимости

- **Библиотеки времени выполнения: не требуются (статическая компоновка MinGW-w64)**
- **Файловая система**: NTFS, FAT32 или exFAT
- **Права доступа**: стандартные пользовательские права

5.3 Описание интерфейса пользователя

5.3.1 Режим зала обслуживания

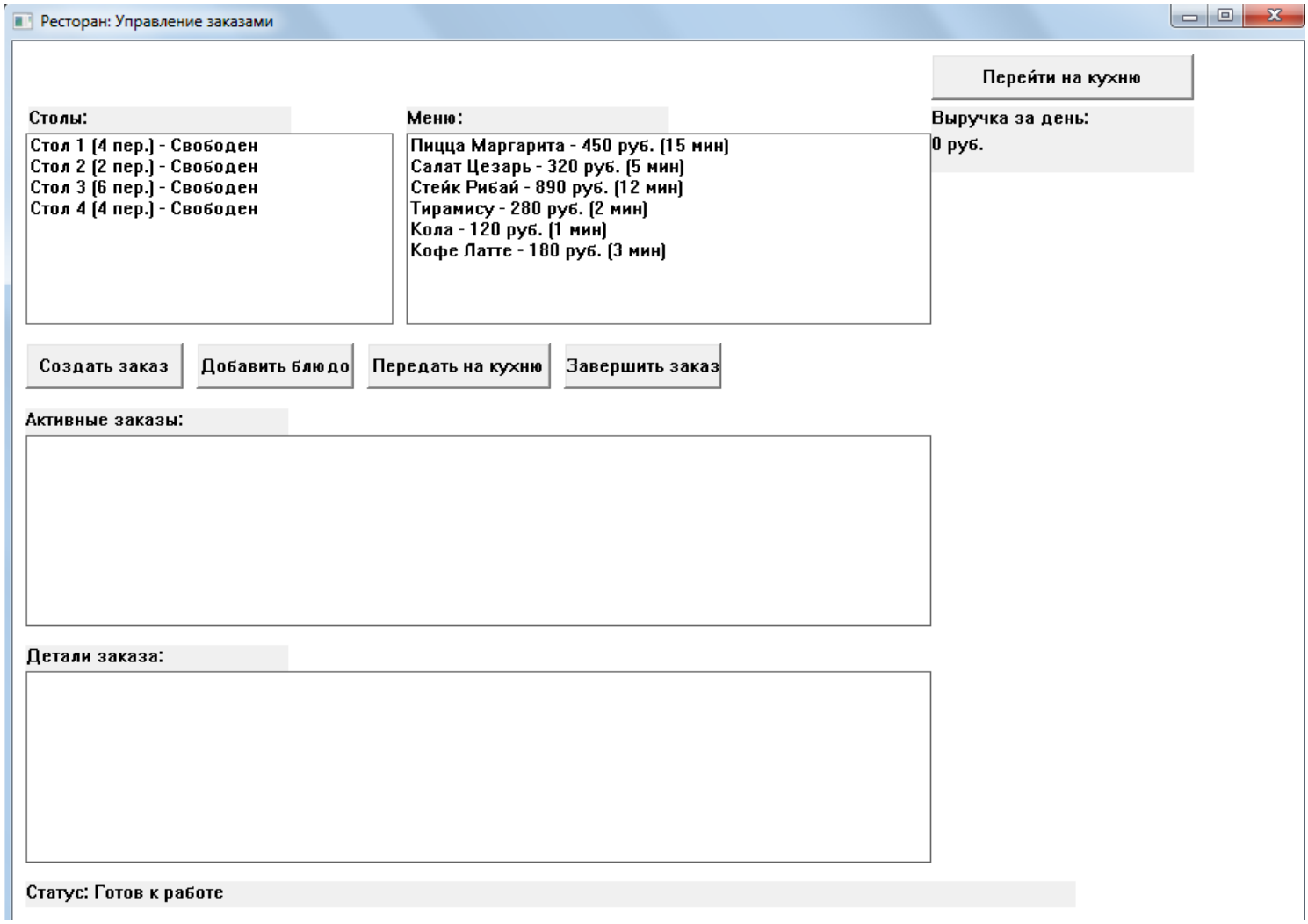


Рисунок 3 - Режим зала обслуживания

Элементы управления в режиме зала:

1. **Кнопка "Перейти на кухню"** – переход в режим управления кухней
2. **Список столов** – отображение всех столов с их статусами
3. **Список меню** – все доступные блюда с ценами
4. **Кнопки управления**:
   - "Создать заказ" – создание нового заказа для выбранного стола
   - "Добавить блюдо" – добавление выбранного блюда в выбранный заказ
   - "Передать на кухню" – отправка заказа на приготовление
   - "Завершить заказ" – завершение и оплата заказа
5. **Список активных заказов** – все незавершенные заказы
6. **Детали заказа** – подробная информация о выбранном заказе
7. **Строка статуса** – информация о последнем действии

5.3.2 Режим кухни

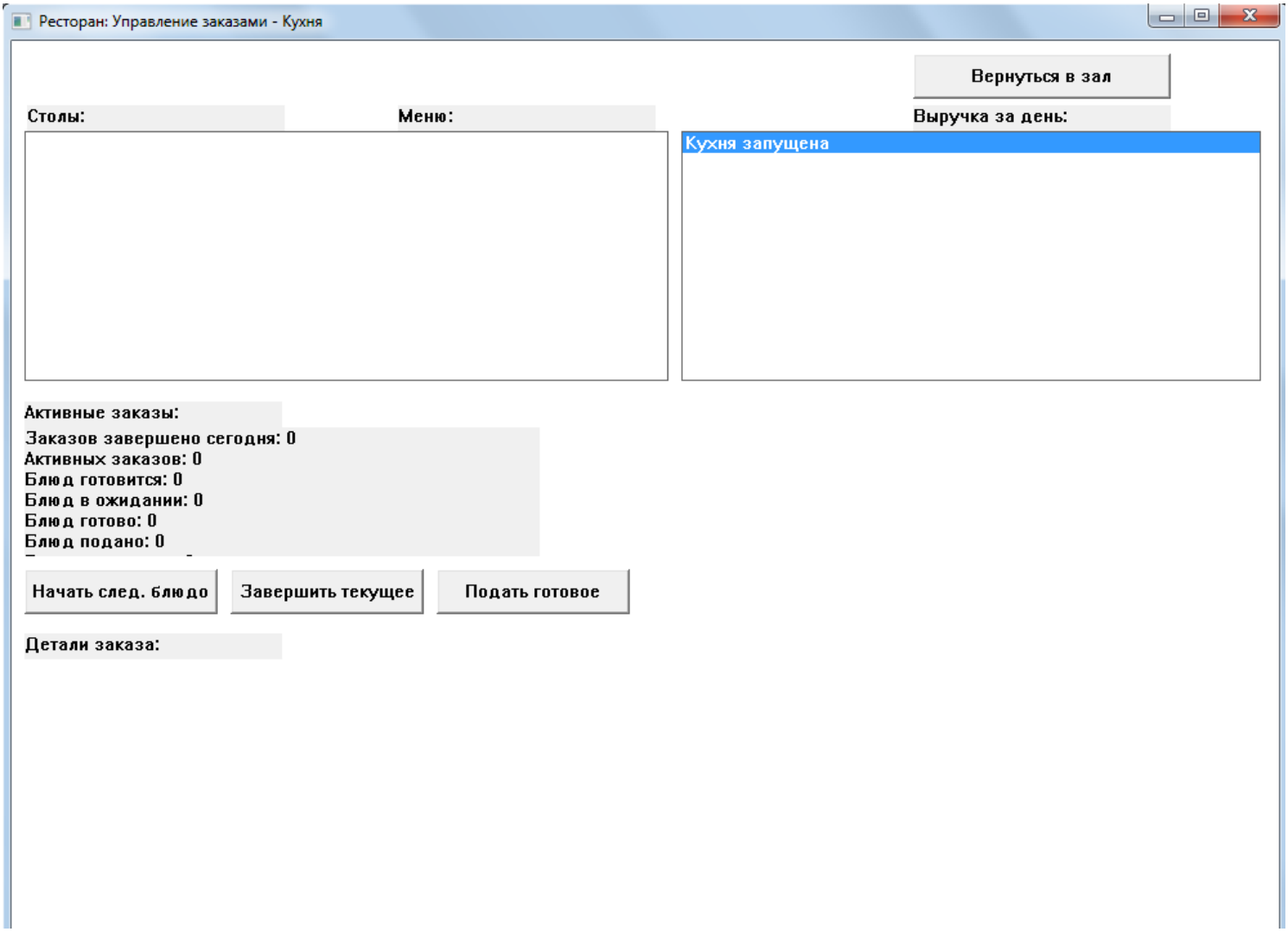


Рисунок 3 - Режим кухни

5.4 Работа с программой

5.4.1 Создание нового заказа

Шаги:

1. В режиме зала выберите свободный стол из списка "Столы"
2. Нажмите кнопку **"Создать заказ"**
3. В строке статуса появится сообщение: "Создан новый заказ для стола X"

Результат:

- Стол меняет статус на "Занят"
- В списке "Активные заказы" появляется новый заказ со статусом "Новый"
- Заказу автоматически присваивается уникальный номер

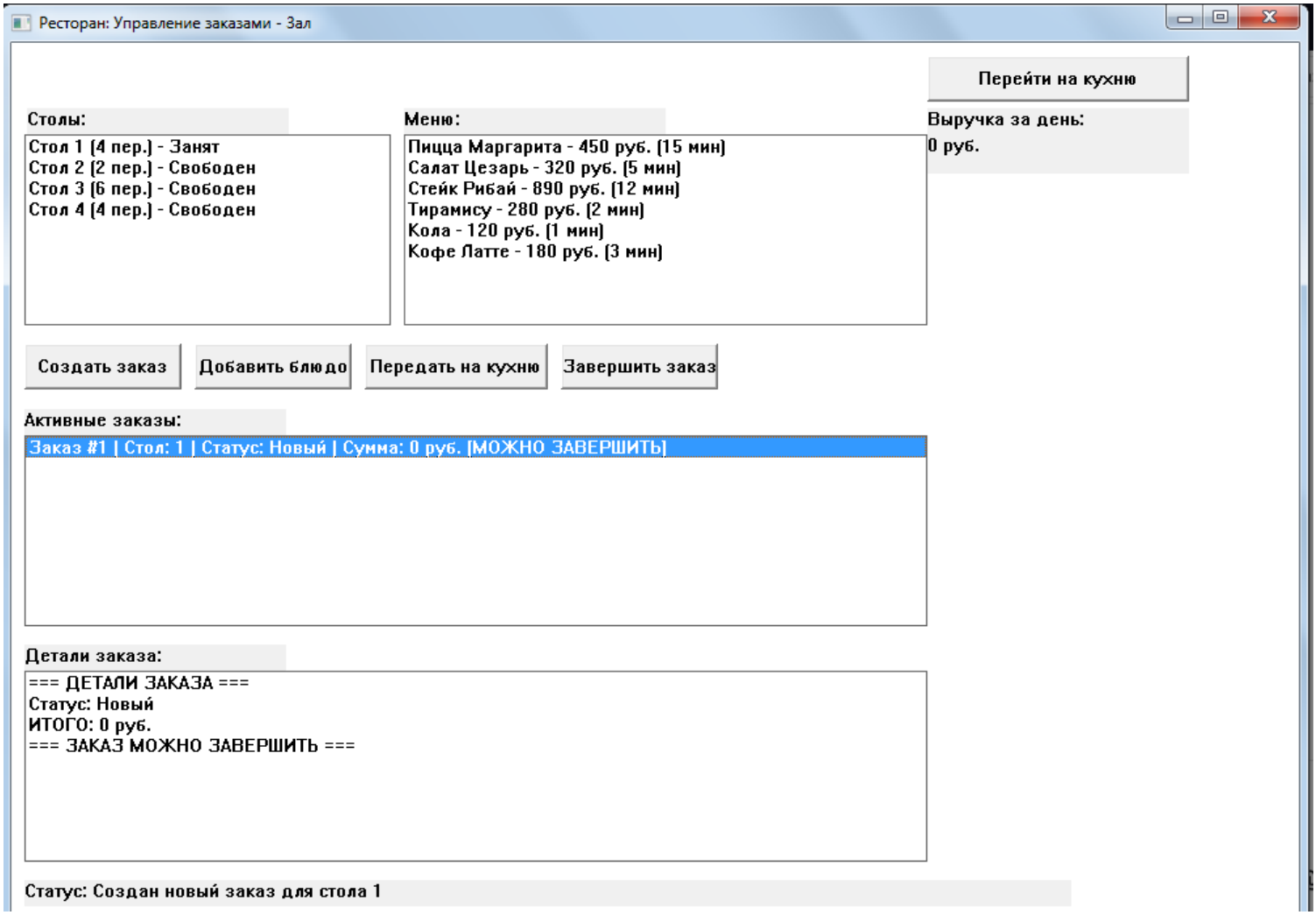


Рисунок 4 – создание заказа

5.4.2 Добавление блюд в заказ

Шаги:

1. Выберите заказ из списка "Активные заказы"
2. Выберите блюдо из списка "Меню"
3. Нажмите кнопку **"Добавить блюдо"**

Результат:

- Блюдо добавляется в заказ
- Сумма заказа обновляется
- В деталях заказа появляется добавленное блюдо со статусом "В ожидании"

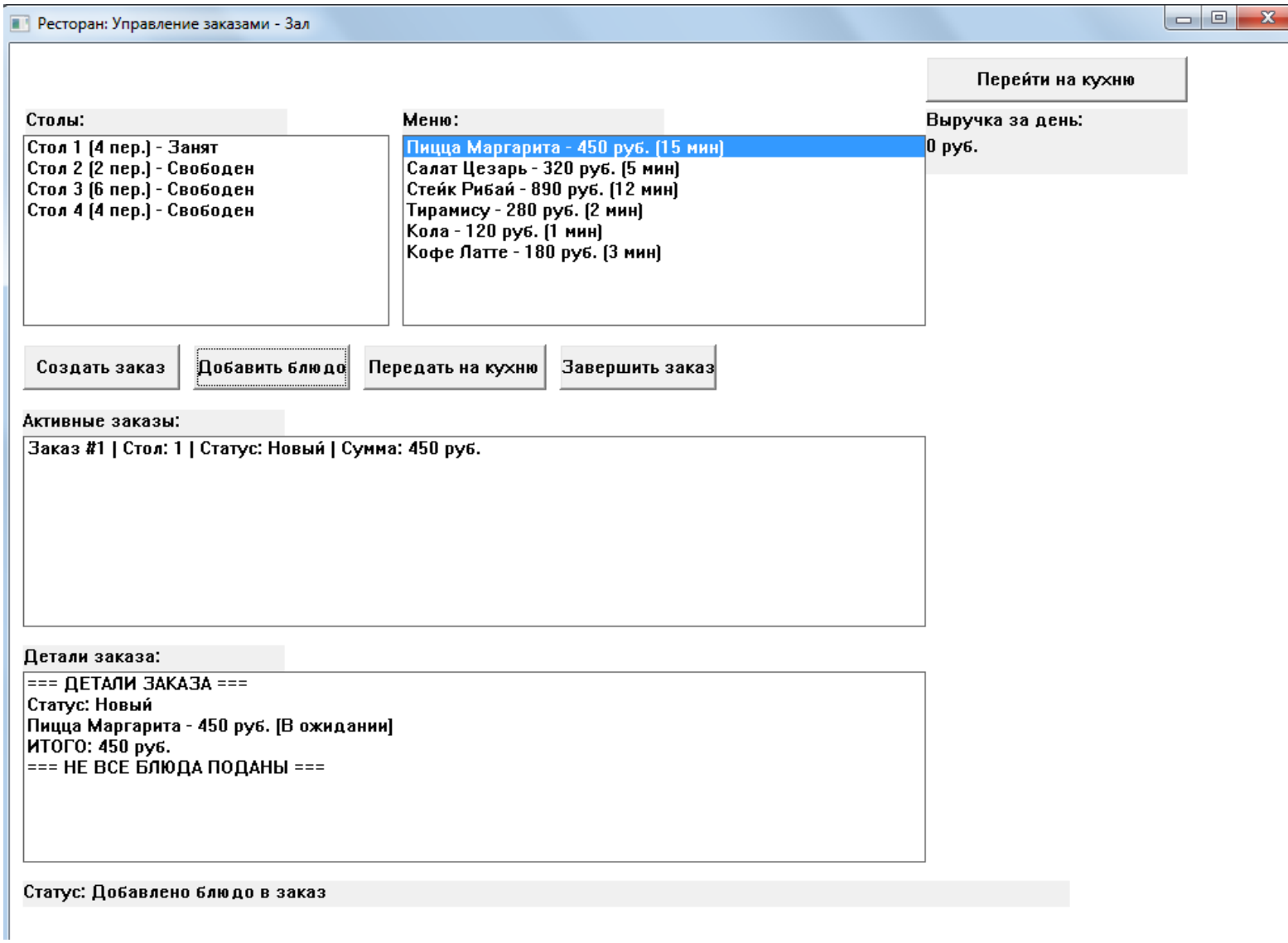


Рисунок 5 – Добавление блюда

5.4.3 Передача заказа на кухню

Шаги:

1. Выберите заказ из списка "Активные заказов"
2. Убедитесь, что в заказ добавлены блюда
3. Нажмите кнопку **"Передать на кухню"**

Результат:

- Статус заказа меняется на "Передан на кухню"
- Заказ появляется в списке заказов на кухне
- Автоматически начинается приготовление первого блюда
- В логе кухни появляются соответствующие записи

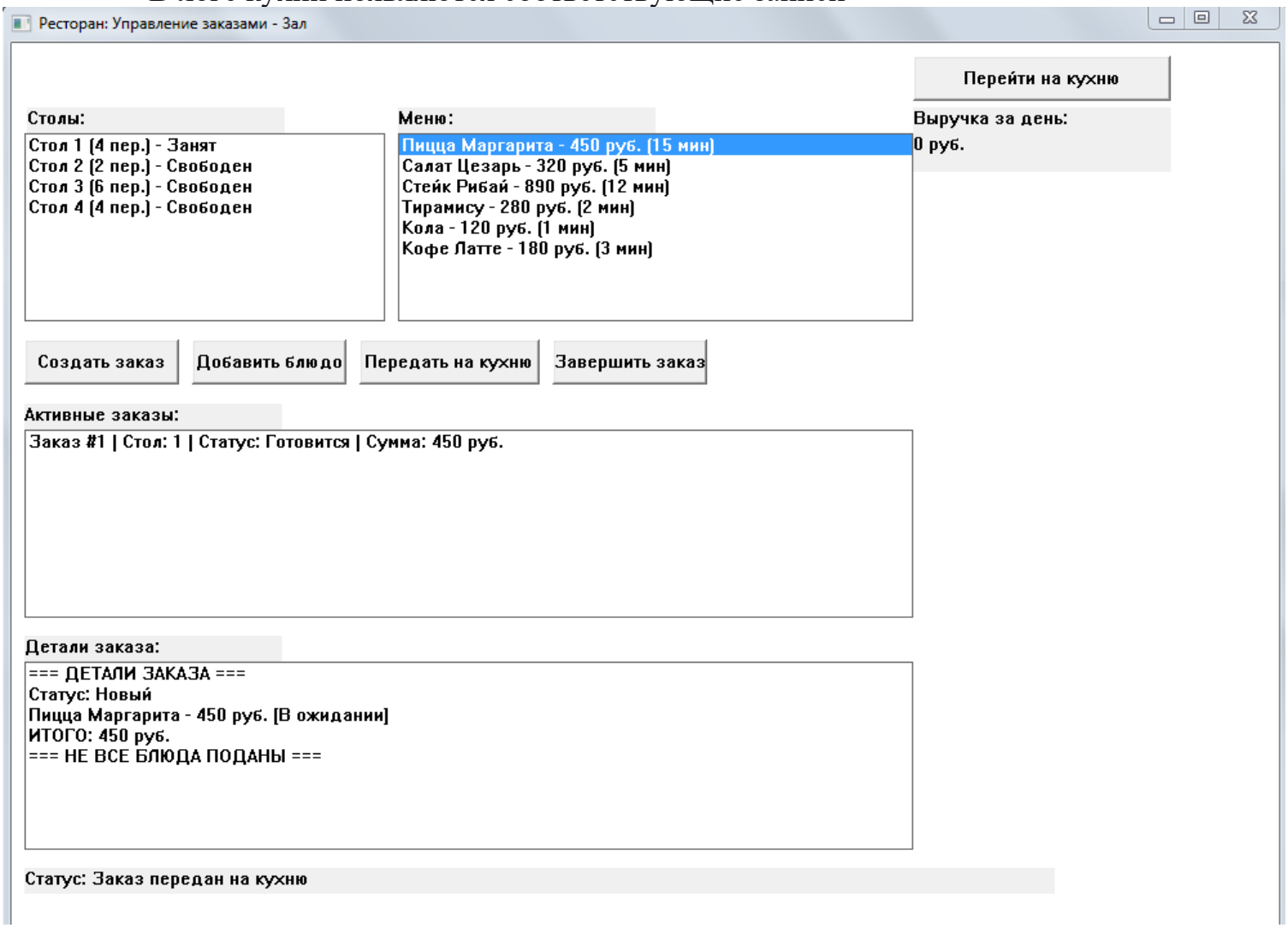


Рисунок 6 – Передача заказа на кухню

5.4.4 Работа на кухне

Начало приготовления:

1. Перейдите в режим кухни (кнопка "Перейти на кухню")
2. Выберите заказ из списка "Заказы на кухне"
3. Нажмите **"Начать след. блюдо"** или система сделает это автоматически

Завершение приготовления:

1. После приготовления блюда выберите заказ
2. Нажмите **"Завершить текущее"**
3. Система автоматически начнет следующее блюдо (если есть)

Подача блюд:

1. Когда блюдо готово, выберите заказ
2. Нажмите **"Подать готовое"**
3. Блюдо отмечается как поданное

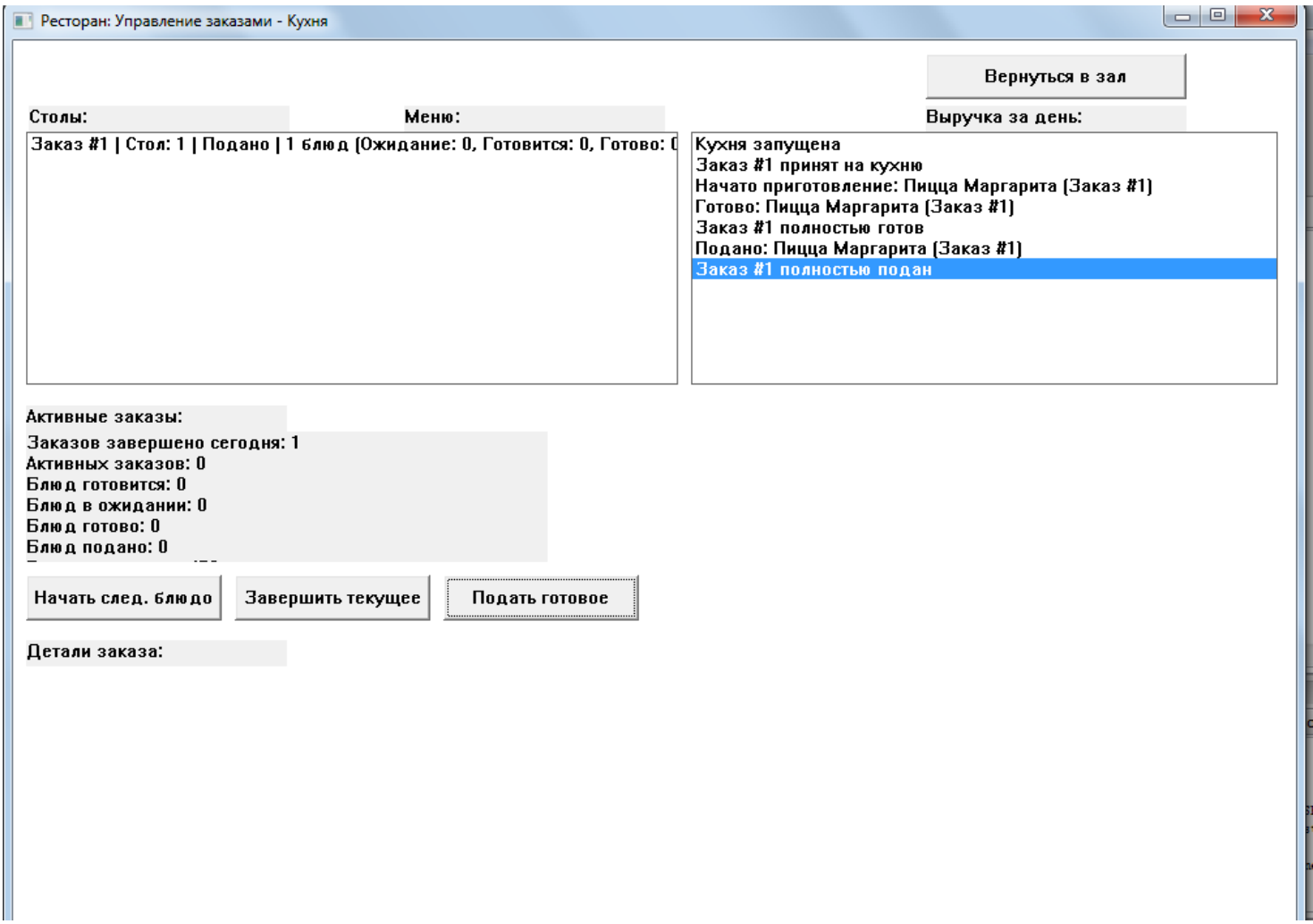


Рисунок 7 – подача заказа

5.4.5 Завершение заказа

Шаги:

1. Вернитесь в режим зала
2. Выберите заказ, у которого все блюда поданы
3. Нажмите **"Завершить заказ"**

Результат:

- Статус заказа меняется на "Завершен"
- Стол освобождается
- Сумма заказа добавляется к общей выручке
- Заказ исчезает из активных

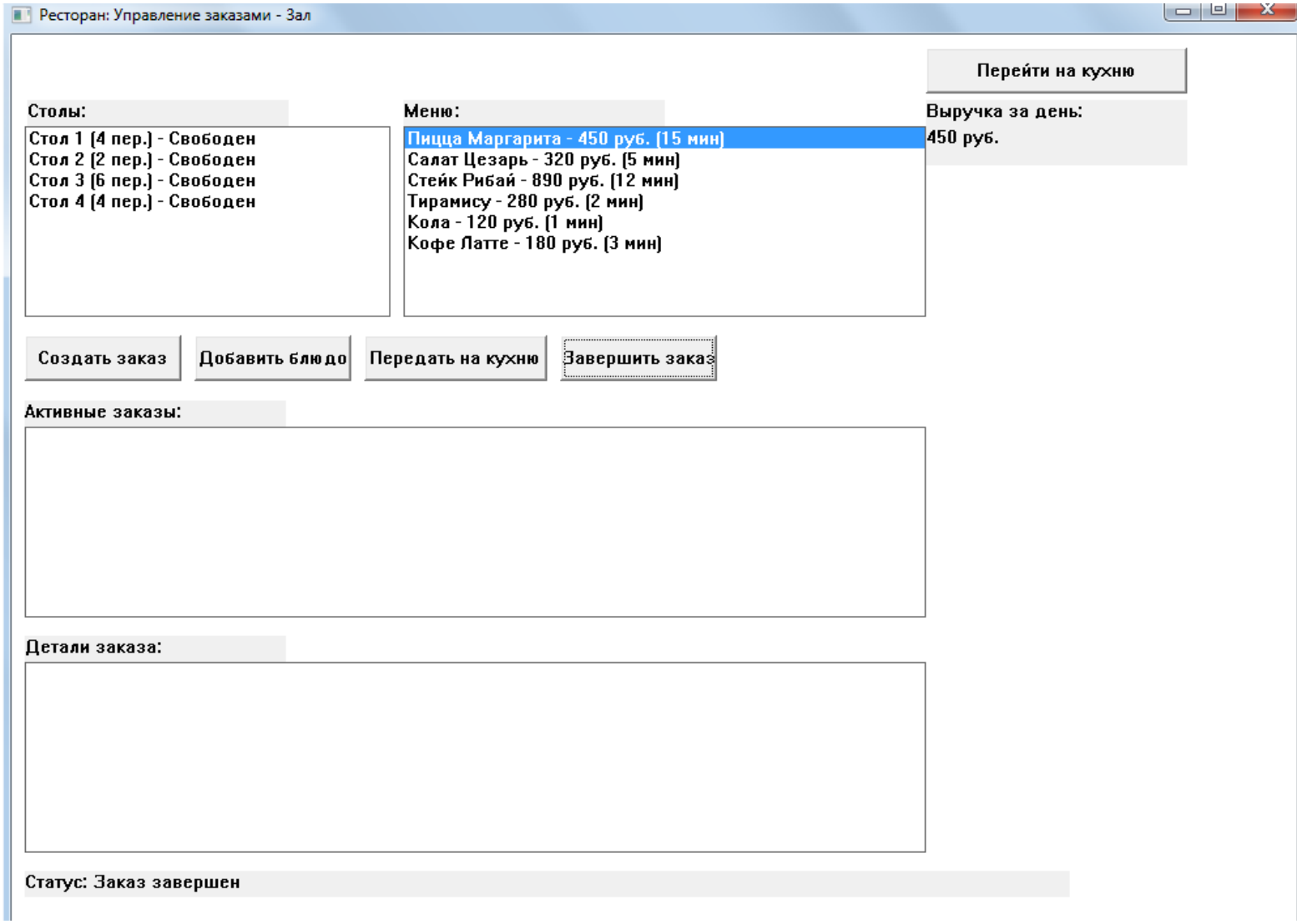


Рисунок 8 – завершение заказа.

5.4.6 Мониторинг статистики

Просмотр статистики:

1. В режиме кухни смотрите раздел "Статистика кухни"
2. В режиме зала смотрите показатель "Выручка за день"

Информация включает:

- Количество завершенных заказов
- Активные заказы
- Статусы блюд (готовятся, ожидают, готовы, поданы)
- Общая выручка за день

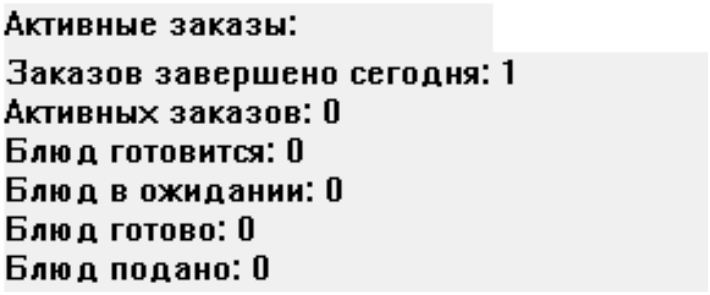


Рисунок 9 – статистика кухни.



Рисунок 10 – выручка за день.

5.5 Описание контрольного примера

5.5.1 Тестовый сценарий

Ситуация: Клиенты садятся за стол №3 (6 персон) и делают заказ.

Шаг 1: Создание заказа

1. В списке столов выбираем "Стол 3 (6 пер.)"
2. Нажимаем "Создать заказ"
3. *Результат*: Создан заказ #1 для стола 3, статус "Новый"

Шаг 2: Добавление блюд

1. Выбираем заказ #1 в активных заказах
2. Из меню выбираем:

- "Пицца Маргарита" (450 руб.)
- "Салат Цезарь" (320 руб.)
- "Кола" (120 руб.) ×2

3. Каждый раз нажимаем "Добавить блюдо"
4. *Результат*: В заказе 4 позиции, сумма 1010 руб.

Шаг 3: Передача на кухню

1. Выбираем заказ #1
2. Нажимаем "Передать на кухню"
3. *Результат*: Статус меняется на "Передан на кухню"

Шаг 4: Работа кухни (автоматическая)

1. Система автоматически начинает готовить "Пицца Маргарита"
2. После завершения пиццы система начинает "Салат Цезарь"
3. После завершения салата начинаются напитки

Шаг 5: Подача блюд

1. По готовности каждого блюда на кухне нажимаем "Подать готовое"
2. После подачи всех блюд заказ считается полностью поданным

Шаг 6: Завершение заказа

1. В зале выбираем заказ #1
2. Нажимаем "Завершить заказ"
3. *Результат*: Стол освобождается, выручка увеличивается на 1010 руб.

5.5.2 Проверка результатов

Ожидаемые результаты:

1. **Статистика кухни**:
   - Заказов завершено: 1
   - Выручка: 1010 руб.
   - Все показатели блюд: 0 (все поданы)
2. **Состояние зала**:
   - Стол 3: свободен
   - Активных заказов: 0
   - Выручка за день: 1010 руб.
3. **Лог кухни** (последние записи):
   - Заказ #1 принят на кухню
   - Начато приготовление: Пицца Маргарита
   - Готово: Пицца Маргарита
   - Начато приготовление: Салат Цезарь
   - Готово: Салат Цезарь
   - Подано: Пицца Маргарита
   - Подано: Салат Цезарь
   - Заказ #1 полностью подан

6 ТЕСТИРОВАНИЕ И ОТЛАДКА ПРОГРАММЫ

6.1 Методология тестирования

6.1.1 Виды тестирования

1. **Модульное тестирование** – тестирование отдельных классов и методов
2. **Интеграционное тестирование** – проверка взаимодействия модулей

3. **Системное тестирование** – тестирование всей системы в целом
4. **Приемочное тестирование** – проверка соответствия требованиям пользователя

6.1.2 Инструменты тестирования

1. **Ручное тестирование** – выполнение тестовых сценариев
2. **Отладчик GDB в Code::Blocks – пошаговое выполнение, точки останова**
3. **Статический анализ кода** – проверка на потенциальные ошибки
4. **Тестовые сценарии** – проверка бизнес-логики

6.2 Тестовые сценарии

6.2.1 Сценарий 1: Создание и обработка заказа

Цель: проверка полного цикла обработки заказа.

Шаги:

1. Создать заказ для свободного стола
2. Добавить 3 разных блюда
3. Передать заказ на кухню
4. На кухне: начать, завершить и подать все блюда
5. В зале: завершить заказ

Ожидаемый результат: Все статусы изменяются корректно, выручка подсчитывается верно.

6.2.2 Сценарий 2: Обработка нескольких заказов

Цель: проверка работы системы с несколькими одновременными заказами.

Шаги:

1. Создать 3 заказа для разных столов
2. Добавить в каждый по 2-3 блюда
3. Передать все на кухню
4. На кухне поочередно обрабатывать блюда
5. Завершать заказы по мере готовности

Ожидаемый результат: Система корректно управляет очередностью, не теряет данные.

6.2.3 Сценарий 3: Обработка ошибок

Цель: проверка устойчивости к некорректным действиям.

Шаги:

1. Попытаться создать заказ для занятого стола
2. Попытаться добавить блюдо без выбора заказа
3. Попытаться передать на кухню пустой заказ
4. Попытаться завершить незаконченный заказ

Ожидаемый результат: Система предотвращает некорректные действия, показывает сообщения.

6.3 Результаты тестирования

6.3.1 Функциональное тестирование

Таблица 1 – Функциональное тестирование

Функция	**Результат**	**Примечания**
Создание заказа	Успешно	Проверены все валидации
Добавление блюд	Успешно	Корректный подсчет суммы
Передача на кухню	Успешно	Автоматический старт приготовления
Приготовление блюд	Успешно	Автоматическая очередь работает
Подача блюд	Успешно	Контроль полной подачи
Завершение заказа	Успешно	Освобождение стола, учет выручки
Переключение режимов	Успешно	Сохранение данных между переключениями
Статистика	Успешно	Корректный подсчет всех показателей

6.3.2 Нефункциональное тестирование

Таблица 2 – Нефункциональное тестирование

Характеристика	**Результат**	**Метрика**
Производительность	Высокая	100+ заказов обрабатываются мгновенно
Надежность	Высокая	Обработка всех исключительных ситуаций
Удобство использования	Хорошее	Интуитивный интерфейс, минимальное обучение
Совместимость	Полная	Windows 7-11, x86/x64

6.3.3 Найденные и исправленные ошибки

1. **Ошибка**: Некорректный подсчет выручки при одновременных заказах
   Решение: Добавлена синхронизация доступа к данным
2. **Ошибка**: Возможность создать заказ для занятого стола
   Решение: Добавлена проверка статуса стола перед созданием заказа
3. **Ошибка**: Потеря фокуса при переключении между режимами
   Решение: Исправлена логика показа/скрытия элементов
4. **Ошибка**: Некорректное отображение времени в логе
   Решение: Исправлено форматирование временных меток

7 ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан программный продукт «Ресторан: Управление заказами» для автоматизации процессов обработки заказов в заведениях общественного питания.

7.1 Достигнутые результаты

1. **Полнофункциональная система**:
   - Реализованы все запланированные функции управления заказами
   - Создана эффективная система координации между залом и кухней
   - Разработан интуитивно понятный графический интерфейс
2. **Качественная реализация**:
   - Соблюдены принципы объектно-ориентированного программирования
   - Использованы современные возможности C++17 и STL
   - Обеспечена надежность и отказоустойчивость системы
3. **Документация**:
   - Разработана полная техническая документация
   - Создано подробное руководство пользователя
   - Подготовлены тестовые сценарии и методики проверки

7.2 Соответствие требованиям

Программа полностью соответствует требованиям технического задания:

- Все функциональные требования реализованы
- Интерфейс соответствует эргономическим требованиям
- Обеспечена надежность работы и защита от ошибок
- Реализована эффективная бизнес-логика

7.3 Научно-практическая значимость

1. **Научная значимость**:
   - Исследованы и применены принципы объектно-ориентированного проектирования
   - Разработана архитектура системы реального времени с разделением ответственности
   - Изучены особенности разработки приложений с использованием Windows API
2. **Практическая значимость**:
   - Программа может использоваться в реальных заведениях общественного питания
   - Сокращает время обработки заказов и уменьшает количество ошибок

- Повышает эффективность работы персонала и удовлетворенность клиентов
- Обеспечивает точный учет финансовых показателей

7.4 Перспективы развития

1. **Доработки и улучшения**:
   - Добавление модуля управления персоналом и сменами
   - Интеграция с фискальными регистраторами для печати чеков
   - Поддержка сетевого режима для нескольких терминалов
   - Добавление модуля складского учета и закупок
2. **Оптимизации**:
   - Переход на базу данных для хранения информации
   - Добавление системы резервного копирования
   - Улучшение производительности при больших объемах данных
3. **Расширение функционала**:
   - Модуль онлайн-заказов и доставки
   - Система лояльности и дисконтных программ
   - Мобильное приложение для официантов
   - Аналитика и прогнозирование продаж

7.5 Выводы

Разработанный программный продукт представляет собой законченное решение для автоматизации обработки заказов в ресторанах и заведениях общественного питания. Программа успешно прошла все этапы тестирования и готова к использованию.

Основные преимущества системы:

- Простота использования и минимальное обучение персонала
- Надежность работы и защита от ошибок
- Эффективная координация между залом и кухней
- Автоматизация рутинных операций
- Точный финансовый учет и отчетность
- Минимальные системные требования

Работа продемонстрировала умение применять теоретические знания на практике, навыки объектно-ориентированного проектирования и программирования на C++ с использованием Windows API.

8 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

8.1 Нормативные документы

1. ГОСТ 34.003-90 "Автоматизированные системы. Термины и определения"
2. ГОСТ 34.601-90 "Автоматизированные системы. Стадии создания"
3. ГОСТ 19.101-77 "Единая система программной документации. Виды программ и программных документов"

8.2 Книги и учебные пособия

1. Страуструп Б. Язык программирования C++. – 4-е изд. – М.: ДМК Пресс, 2019. – 1136 с.
2. Петцольд Ч. Программирование для Windows 95. – М.: БИНОМ, 1996. – 848 с.
3. Рихтер Д. Windows для профессионалов. – СПб.: Питер, 2018. – 752 с.
4. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб.: Питер, 2020. – 366 с.
5. Макконнелл С. Совершенный код. – 2-е изд. – М.: Русская редакция, 2019. – 896 с.
6. Липпман С., Лажойе Ж., Му Б. Язык программирования C++. Базовый курс. – 5-е изд. – М.: Вильямс, 2021. – 1120 с.

8.3 Статьи и научные публикации

1. Sommerville I. Software Engineering. – 10th ed. – Pearson, 2015.
2. Pressman R.S. Software Engineering: A Practitioner's Approach. – 8th ed. – McGraw-Hill, 2014.
3. Larman C. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. – 3rd ed. – Prentice Hall, 2004.

8.4 Интернет-ресурсы

1. Microsoft Docs: Windows API Reference – https://docs.microsoft.com/en-us/windows/win32/api/
2. C++ Reference – https://en.cppreference.com/w/
3. Stack Overflow – https://stackoverflow.com/
4. GitHub – https://github.com/

8.5 Программное обеспечение

1. Code::Blocks 25.03 (компилятор MinGW-w64 GCC 14.2)
2. Заголовочные файлы и библиотеки Windows API из состава MinGW-w64
3. Git для контроля версий
4. Doxygen для генерации документации

9 ПРИЛОЖЕНИЯ

Приложение А. Листинг программы

```
#include <windows.h>
#include <commctrl.h>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
#include <map>
#include <ctime>

#pragma comment(linker, "/subsystem:windows /entry:mainCRTStartup")
#pragma comment(lib, "comctl32.lib")

using namespace std;

// ==================== КЛАССЫ ДАННЫХ ====================
class MenuItem {
public:
    int id;
    string name;
    string category;
    double price;
    int prepTime;
    string kitchenNotes;

    MenuItem(int i = 0, string n = "", string cat = "", double p = 0.0, int time = 10,
string notes = "")
        : id(i), name(n), category(cat), price(p), prepTime(time), kitchenNotes(notes)
{}
};

class Table {
public:
    int number;
    int capacity;
    bool isOccupied;
    string location;

    Table(int num = 0, int cap = 4, string loc = "Зал")
        : number(num), capacity(cap), isOccupied(false), location(loc) {}
};
```

```
class OrderItem {
public:
    MenuItem item;
    string status;
    time_t startTime;
    time_t finishTime;
    string chefNotes;

    OrderItem(const MenuItem& menuItem)
        : item(menuItem), status("В ожидании"), startTime(0), finishTime(0), chefNotes("") {}
};

class Order {
public:
    int id;
    int tableNumber;
    vector<OrderItem> items;
    string status;
    time_t createdTime;
    time_t kitchenAcceptTime;
    string waiterNotes;
    int currentCookingIndex; // Новое поле: индекс текущего готовящегося блюда

    Order(int i = 0, int tableNum = 0)
        : id(i), tableNumber(tableNum), status("Новый"), createdTime(time(0)),
          kitchenAcceptTime(0), waiterNotes(""), currentCookingIndex(-1) {}

    double getTotal() const {
        double total = 0;
        for (const auto& orderItem : items) {
            total += orderItem.item.price;
        }
        return total;
    }

    void addItem(const MenuItem& item) {
        items.push_back(OrderItem(item));
    }

    bool isReady() const {
        for (const auto& orderItem : items) {
            if (orderItem.status != "Готово" && orderItem.status != "Подано") {
```

```
            return false;
        }
    }
    return true;
}

bool isServed() const {
    for (const auto& orderItem : items) {
        if (orderItem.status != "Подано") {
            return false;
        }
    }
    return true;
}

bool canBeCompleted() const {
    return isServed() && status != "Завершен";
}

// Новый метод: получение индекса следующего блюда для приготовления
int getNextItemToCook() const {
    for (int i = 0; i < items.size(); i++) {
        if (items[i].status == "В ожидании") {
            return i;
        }
    }
    return -1; // Нет блюд для приготовления
}

// Новый метод: получение индекса следующего готового блюда для подачи
int getNextItemToServe() const {
    for (int i = 0; i < items.size(); i++) {
        if (items[i].status == "Готово") {
            return i;
        }
    }
    return -1; // Нет готовых блюд
}
};

// ==================== МОДУЛЬ КУХНИ И ФИНАНСОВ ====================
class KitchenManager {
```

```
private:
    vector<Order*> activeOrders;
    vector<string> kitchenLog;
    string currentChef;
    int ordersCompletedToday;
    double dailyRevenue;

public:
    KitchenManager() : ordersCompletedToday(0), dailyRevenue(0.0) {
        currentChef = "Шеф-повар Иванов";
        kitchenLog.push_back("Кухня запущена");
    }

    void acceptOrder(Order* order) {
        if (order && order->status == "Новый") {
            order->status = "Передан на кухню";
            order->kitchenAcceptTime = time(0);
            activeOrders.push_back(order);

            string logEntry = "Заказ #" + to_string(order->id) + " принят на кухню";
            kitchenLog.push_back(logEntry);
        }
    }

    void startCookingItem(Order* order, int itemIndex) {
        if (order && itemIndex >= 0 && itemIndex < order->items.size()) {
            order->items[itemIndex].status = "Готовится";
            order->items[itemIndex].startTime = time(0);
            order->status = "Готовится";
            order->currentCookingIndex = itemIndex;

            string logEntry = "Начато приготовление: " + order-
>items[itemIndex].item.name +
                              " (Заказ #" + to_string(order->id) + ")";
            kitchenLog.push_back(logEntry);
        }
    }

    // Новый метод: автоматический старт приготовления следующего блюда
    bool startNextItem(Order* order) {
        if (!order) return false;

        int nextIndex = order->getNextItemToCook();
        if (nextIndex >= 0) {
```

```
            startCookingItem(order, nextIndex);
            return true;
        }
        return false;
    }

    void completeItem(Order* order, int itemIndex, const string& chefNotes = "") {
        if (order && itemIndex >= 0 && itemIndex < order->items.size()) {
            order->items[itemIndex].status = "Готово";
            order->items[itemIndex].finishTime = time(0);
            order->items[itemIndex].chefNotes = chefNotes;
            order->currentCookingIndex = -1;

            string logEntry = "Готово: " + order->items[itemIndex].item.name +
                              " (Заказ #" + to_string(order->id) + ")";
            kitchenLog.push_back(logEntry);

            // Автоматически начинаем готовить следующее блюдо, если есть
            if (startNextItem(order)) {
                logEntry = "Автоматически начато следующее блюдо (Заказ #" +
to_string(order->id) + ")";
                kitchenLog.push_back(logEntry);
            }

            if (order->isReady()) {
                order->status = "Готов";
                string completeLog = "Заказ #" + to_string(order->id) + " полностью
готов";
                kitchenLog.push_back(completeLog);
            }
        }
    }

    void markItemServed(Order* order, int itemIndex) {
        if (order && itemIndex >= 0 && itemIndex < order->items.size()) {
            order->items[itemIndex].status = "Подано";

            string logEntry = "Подано: " + order->items[itemIndex].item.name +
                              " (Заказ #" + to_string(order->id) + ")";
            kitchenLog.push_back(logEntry);

            bool allServed = true;
            for (const auto& item : order->items) {
                if (item.status != "Подано") {
```

```
                allServed = false;
                break;
            }
        }

        if (allServed) {
            order->status = "Подано";
            string serveLog = "Заказ #" + to_string(order->id) + " полностью подан";
            kitchenLog.push_back(serveLog);

            // Добавляем в выручку при полной подаче заказа
            dailyRevenue += order->getTotal();

            auto it = find(activeOrders.begin(), activeOrders.end(), order);
            if (it != activeOrders.end()) {
                activeOrders.erase(it);
                ordersCompletedToday++;
            }
        }
    }
}

// Новый метод: автоматическая подача следующего готового блюда
bool serveNextItem(Order* order) {
    if (!order) return false;

    int nextIndex = order->getNextItemToServe();
    if (nextIndex >= 0) {
        markItemServed(order, nextIndex);
        return true;
    }
    return false;
}

vector<Order*> getActiveKitchenOrders() const {
    return activeOrders;
}

vector<string> getKitchenLog(int maxEntries = 20) const {
    if (kitchenLog.size() <= maxEntries) {
        return kitchenLog;
    }
    return vector<string>(kitchenLog.end() - maxEntries, kitchenLog.end());
```

```
    }

    map<string, double> getKitchenStatistics() const {
        map<string, double> stats;
        stats["orders_completed_today"] = ordersCompletedToday;
        stats["active_orders"] = activeOrders.size();
        stats["daily_revenue"] = dailyRevenue;

        int cookingItems = 0;
        int waitingItems = 0;
        int readyItems = 0;
        int servedItems = 0;

        for (const auto& order : activeOrders) {
            for (const auto& item : order->items) {
                if (item.status == "Готовится") cookingItems++;
                if (item.status == "В ожидании") waitingItems++;
                if (item.status == "Готово") readyItems++;
                if (item.status == "Подано") servedItems++;
            }
        }

        stats["items_cooking"] = cookingItems;
        stats["items_waiting"] = waitingItems;
        stats["items_ready"] = readyItems;
        stats["items_served"] = servedItems;

        return stats;
    }

    double getDailyRevenue() const {
        return dailyRevenue;
    }
};

// ==================== ОСНОВНОЙ МЕНЕДЖЕР ДАННЫХ ====================
class DataManager {
private:
    vector<Table> tables;
    vector<MenuItem> menu;
    vector<Order> orders;
    int nextOrderId;
    KitchenManager kitchen;
```

```
public:
    DataManager() : nextOrderId(1) {
        initializeData();
    }

    void initializeData() {
        tables.push_back(Table(1, 4, "Основной зал"));
        tables.push_back(Table(2, 2, "Основной зал"));
        tables.push_back(Table(3, 6, "VIP зал"));
        tables.push_back(Table(4, 4, "Терраса"));

        menu.push_back(MenuItem(1, "Пицца Маргарита", "Основные блюда",
450, 15, "Печь 250°C"));
        menu.push_back(MenuItem(2, "Салат Цезарь", "Салаты", 320, 5, "Соус
цезарь"));
        menu.push_back(MenuItem(3, "Стейк Рибай", "Основные блюда", 890, 12,
"Medium rare"));
        menu.push_back(MenuItem(4, "Тирамису", "Десерты", 280, 2,
"Охлажденный"));
        menu.push_back(MenuItem(5, "Кола", "Напитки", 120, 1, "Со льдом"));
        menu.push_back(MenuItem(6, "Кофе Латте", "Напитки", 180, 3, "Двойной
эспрессо"));
    }

    vector<Table> getTables() const { return tables; }
    vector<MenuItem> getMenu() const { return menu; }
    vector<Order> getOrders() const { return orders; }
    KitchenManager& getKitchen() { return kitchen; }

    Table* getTableByNumber(int number) {
        for (auto& table : tables) {
            if (table.number == number) return &table;
        }
        return nullptr;
    }

    MenuItem* getItemById(int id) {
        for (auto& item : menu) {
            if (item.id == id) return &item;
        }
        return nullptr;
    }
```

```
Order* getOrderById(int id) {
    for (auto& order : orders) {
        if (order.id == id) return &order;
    }
    return nullptr;
}

int createOrder(int tableNumber, const string& waiterNotes = "") {
    Order newOrder(nextOrderId++, tableNumber);
    newOrder.waiterNotes = waiterNotes;
    orders.push_back(newOrder);

    Table* table = getTableByNumber(tableNumber);
    if (table) table->isOccupied = true;

    return newOrder.id;
}

void addItemToOrder(int orderId, int itemId) {
    Order* order = getOrderById(orderId);
    MenuItem* item = getItemById(itemId);

    if (order && item) {
        order->addItem(*item);
    }
}

void sendOrderToKitchen(int orderId) {
    Order* order = getOrderById(orderId);
    if (order && order->status == "Новый") {
        kitchen.acceptOrder(order);
        // Автоматически начинаем готовить первое блюдо
        kitchen.startNextItem(order);
    }
}

bool completeOrder(int orderId) {
    Order* order = getOrderById(orderId);
    if (order && order->canBeCompleted()) {
        order->status = "Завершен";
        Table* table = getTableByNumber(order->tableNumber);
        if (table) table->isOccupied = false;
        return true;
    }
```

```
        return false;
    }

    vector<Order> getActiveOrders() const {
        vector<Order> active;
        for (const auto& order : orders) {
            if (order.status != "Завершен") {
                active.push_back(order);
            }
        }
        return active;
    }

    vector<Order> getKitchenOrders() const {
        vector<Order> kitchenOrders;
        for (const auto& order : orders) {
            if (order.status == "Передан на кухню" || order.status == "Готовится" ||
order.status == "Готов" || order.status == "Подано") {
                kitchenOrders.push_back(order);
            }
        }
        return kitchenOrders;
    }

    double getTotalRevenue() const {
        return kitchen.getDailyRevenue();
    }
};

//          ====================          ГЛОБАЛЬНЫЕ          ПЕРЕМЕННЫЕ
====================
DataManager dataManager;
HWND hMainWindow;

HWND hTableList, hMenuList, hOrderList, hActiveOrdersList;
HWND          hCreateOrderBtn,          hAddItemBtn,          hCompleteOrderBtn,
hSendToKitchenBtn;
HWND hStatusText, hSwitchToKitchenBtn, hRevenueText;

HWND hKitchenOrdersList, hKitchenLogList, hKitchenStatsText;
HWND hStartCookingBtn, hCompleteItemBtn, hMarkServedBtn;
HWND hSwitchToHallBtn;

bool isKitchenMode = false;
```

```
// ==================== ПРОТОТИПЫ ФУНКЦИЙ ====================
void CreateHallControls(HWND hwnd);
void CreateKitchenControls(HWND hwnd);
void ShowHallInterface();
void ShowKitchenInterface();
void RefreshTableList();
void RefreshMenuList();
void RefreshActiveOrdersList();
void RefreshKitchenOrdersList();
void RefreshKitchenLog();
void RefreshKitchenStats();
void RefreshOrderDetails(int orderId);
void RefreshRevenue();
void UpdateStatus(const string& message);
void UpdateKitchenStatus(const string& message);
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam);

// ==================== ГЛАВНАЯ ФУНКЦИЯ ====================
int main() {
    HINSTANCE hInstance = GetModuleHandle(NULL);

    WNDCLASS wc = {};
    wc.lpfnWndProc = WindowProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = "RestaurantClass";
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);

    RegisterClass(&wc);

    hMainWindow = CreateWindow(
        "RestaurantClass",
        "Ресторан: Управление заказами",
        WS_OVERLAPPEDWINDOW | WS_VISIBLE,
        CW_USEDEFAULT, CW_USEDEFAULT, 1000, 800,
        NULL, NULL, hInstance, NULL
    );

    ShowWindow(hMainWindow, SW_SHOW);
    UpdateWindow(hMainWindow);
```

```
    MSG msg = {};
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return 0;
}

// ==================== СОЗДАНИЕ ЭЛЕМЕНТОВ ИНТЕРФЕЙСА ====================
void CreateHallControls(HWND hwnd) {
    hSwitchToKitchenBtn = CreateWindow("BUTTON", "Перейти на кухню",
        WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
        700, 10, 200, 35, hwnd, (HMENU)10, NULL, NULL);

    CreateWindow("STATIC", "Столы:", WS_VISIBLE | WS_CHILD,
        12, 50, 200, 20, hwnd, NULL, NULL, NULL);
    hTableList = CreateWindow("LISTBOX", "", WS_VISIBLE | WS_CHILD | WS_BORDER | LBS_NOTIFY,
        10, 70, 280, 150, hwnd, NULL, NULL, NULL);

    CreateWindow("STATIC", "Меню:", WS_VISIBLE | WS_CHILD,
        300, 50, 200, 20, hwnd, NULL, NULL, NULL);
    hMenuList = CreateWindow("LISTBOX", "", WS_VISIBLE | WS_CHILD | WS_BORDER | LBS_NOTIFY,
        300, 70, 400, 150, hwnd, NULL, NULL, NULL);

    hCreateOrderBtn = CreateWindow("BUTTON", "Создать заказ",
        WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
        10, 230, 120, 35, hwnd, (HMENU)1, NULL, NULL);
    hAddItemBtn = CreateWindow("BUTTON", "Добавить блюдо",
        WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
        140, 230, 120, 35, hwnd, (HMENU)2, NULL, NULL);
    hSendToKitchenBtn = CreateWindow("BUTTON", "Передать на кухню",
        WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
        270, 230, 140, 35, hwnd, (HMENU)4, NULL, NULL);
    hCompleteOrderBtn = CreateWindow("BUTTON", "Завершить заказ",
        WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
        420, 230, 120, 35, hwnd, (HMENU)3, NULL, NULL);

    CreateWindow("STATIC", "Активные заказы:", WS_VISIBLE | WS_CHILD,
        10, 280, 200, 20, hwnd, NULL, NULL, NULL);
```

```
    hActiveOrdersList = CreateWindow("LISTBOX", "", WS_VISIBLE |
WS_CHILD | WS_BORDER | LBS_NOTIFY,
        10, 300, 690, 150, hwnd, NULL, NULL, NULL);

    CreateWindow("STATIC", "Детали заказа:", WS_VISIBLE | WS_CHILD,
        10, 460, 200, 20, hwnd, NULL, NULL, NULL);
    hOrderList = CreateWindow("LISTBOX", "", WS_VISIBLE | WS_CHILD |
WS_BORDER,
        10, 480, 690, 150, hwnd, NULL, NULL, NULL);

    // Добавляем отображение выручки
    CreateWindow("STATIC", "Выручка за день:", WS_VISIBLE | WS_CHILD,
        700, 50, 200, 20, hwnd, NULL, NULL, NULL);
    hRevenueText = CreateWindow("STATIC", "0 руб.", WS_VISIBLE |
WS_CHILD,
        700, 70, 200, 30, hwnd, NULL, NULL, NULL);

    hStatusText = CreateWindow("STATIC", "Статус: Готов к работе",
        WS_VISIBLE | WS_CHILD,
        10, 640, 800, 20, hwnd, NULL, NULL, NULL);

    RefreshTableList();
    RefreshMenuList();
    RefreshActiveOrdersList();
    RefreshRevenue();
}

void CreateKitchenControls(HWND hwnd) {
    hSwitchToHallBtn = CreateWindow("BUTTON", "Вернуться в зал",
        WS_CHILD | BS_PUSHBUTTON,
        700, 10, 200, 35, hwnd, (HMENU)11, NULL, NULL);

    CreateWindow("STATIC", "Заказы на кухне:", WS_CHILD,
        10, 50, 200, 20, hwnd, NULL, NULL, NULL);
    hKitchenOrdersList = CreateWindow("LISTBOX", "", WS_CHILD |
WS_BORDER | LBS_NOTIFY,
        10, 70, 500, 200, hwnd, NULL, NULL, NULL);

    CreateWindow("STATIC", "Лог кухни:", WS_CHILD,
        520, 50, 200, 20, hwnd, NULL, NULL, NULL);
    hKitchenLogList = CreateWindow("LISTBOX", "", WS_CHILD |
WS_BORDER,
        520, 70, 450, 200, hwnd, NULL, NULL, NULL);
```

```
    CreateWindow("STATIC", "Статистика кухни:", WS_CHILD,
        10, 280, 200, 20, hwnd, NULL, NULL, NULL);
    hKitchenStatsText = CreateWindow("STATIC", "", WS_CHILD,
        10, 300, 400, 100, hwnd, NULL, NULL, NULL);

    // Изменяем названия кнопок для лучшего понимания
    hStartCookingBtn = CreateWindow("BUTTON", "Начать след. блюдо",
        WS_CHILD | BS_PUSHBUTTON,
        10, 410, 150, 35, hwnd, (HMENU)5, NULL, NULL);
    hCompleteItemBtn = CreateWindow("BUTTON", "Завершить текущее",
        WS_CHILD | BS_PUSHBUTTON,
        170, 410, 150, 35, hwnd, (HMENU)6, NULL, NULL);
    hMarkServedBtn = CreateWindow("BUTTON", "Подать готовое",
        WS_CHILD | BS_PUSHBUTTON,
        330, 410, 150, 35, hwnd, (HMENU)7, NULL, NULL);
}

void ShowHallInterface() {
    ShowWindow(hSwitchToKitchenBtn, SW_SHOW);
    ShowWindow(hTableList, SW_SHOW);
    ShowWindow(hMenuList, SW_SHOW);
    ShowWindow(hCreateOrderBtn, SW_SHOW);
    ShowWindow(hAddItemBtn, SW_SHOW);
    ShowWindow(hSendToKitchenBtn, SW_SHOW);
    ShowWindow(hCompleteOrderBtn, SW_SHOW);
    ShowWindow(hActiveOrdersList, SW_SHOW);
    ShowWindow(hOrderList, SW_SHOW);
    ShowWindow(hStatusText, SW_SHOW);
    ShowWindow(hRevenueText, SW_SHOW);

    ShowWindow(hSwitchToHallBtn, SW_HIDE);
    ShowWindow(hKitchenOrdersList, SW_HIDE);
    ShowWindow(hKitchenLogList, SW_HIDE);
    ShowWindow(hKitchenStatsText, SW_HIDE);
    ShowWindow(hStartCookingBtn, SW_HIDE);
    ShowWindow(hCompleteItemBtn, SW_HIDE);
    ShowWindow(hMarkServedBtn, SW_HIDE);

    SetWindowTextA(hMainWindow, "Ресторан: Управление заказами - Зал");
}

void ShowKitchenInterface() {
    ShowWindow(hSwitchToKitchenBtn, SW_HIDE);
    ShowWindow(hTableList, SW_HIDE);
```

```
    ShowWindow(hMenuList, SW_HIDE);
    ShowWindow(hCreateOrderBtn, SW_HIDE);
    ShowWindow(hAddItemBtn, SW_HIDE);
    ShowWindow(hSendToKitchenBtn, SW_HIDE);
    ShowWindow(hCompleteOrderBtn, SW_HIDE);
    ShowWindow(hActiveOrdersList, SW_HIDE);
    ShowWindow(hOrderList, SW_HIDE);
    ShowWindow(hStatusText, SW_HIDE);
    ShowWindow(hRevenueText, SW_HIDE);

    RefreshKitchenOrdersList();
    RefreshKitchenLog();
    RefreshKitchenStats();

    ShowWindow(hSwitchToHallBtn, SW_SHOW);
    ShowWindow(hKitchenOrdersList, SW_SHOW);
    ShowWindow(hKitchenLogList, SW_SHOW);
    ShowWindow(hKitchenStatsText, SW_SHOW);
    ShowWindow(hStartCookingBtn, SW_SHOW);
    ShowWindow(hCompleteItemBtn, SW_SHOW);
    ShowWindow(hMarkServedBtn, SW_SHOW);

    SetWindowTextA(hMainWindow, "Ресторан: Управление заказами - Кухня");
}

// ==================== ОБНОВЛЕНИЕ ИНТЕРФЕЙСА ====================
void RefreshTableList() {
    SendMessage(hTableList, LB_RESETCONTENT, 0, 0);
    auto tables = dataManager.getTables();
    for (const auto& table : tables) {
        string status = table.isOccupied ? "Занят" : "Свободен";
        string item = "Стол " + to_string(table.number) + " (" + to_string(table.capacity) + " пер.) - " + status;
        SendMessageA(hTableList, LB_ADDSTRING, 0, (LPARAM)item.c_str());
    }
}

void RefreshMenuList() {
    SendMessage(hMenuList, LB_RESETCONTENT, 0, 0);
    auto menu = dataManager.getMenu();
    for (const auto& item : menu) {
        string listItem = item.name + " - " + to_string((int)item.price) + " руб. (" +
```

```
to_string(item.prepTime) + " мин)";
        SendMessageA(hMenuList, LB_ADDSTRING, 0, (LPARAM)listItem.c_str());
    }
}

void RefreshActiveOrdersList() {
    SendMessage(hActiveOrdersList, LB_RESETCONTENT, 0, 0);
    auto orders = dataManager.getActiveOrders();
    for (const auto& order : orders) {
        string canComplete = order.canBeCompleted() ? " [МОЖНО ЗАВЕРШИТЬ]" : "";
        string item = "Заказ #" + to_string(order.id) + " | Стол: " + to_string(order.tableNumber) +
                      " | Статус: " + order.status + " | Сумма: " + to_string((int)order.getTotal()) + " руб." + canComplete;
        SendMessageA(hActiveOrdersList, LB_ADDSTRING, 0, (LPARAM)item.c_str());
    }
}

void RefreshKitchenOrdersList() {
    SendMessage(hKitchenOrdersList, LB_RESETCONTENT, 0, 0);
    auto orders = dataManager.getKitchenOrders();
    for (const auto& order : orders) {
        int cooking = 0, waiting = 0, ready = 0, served = 0;
        for (const auto& item : order.items) {
            if (item.status == "Готовится") cooking++;
            if (item.status == "В ожидании") waiting++;
            if (item.status == "Готово") ready++;
            if (item.status == "Подано") served++;
        }

        string item = "Заказ #" + to_string(order.id) + " | Стол: " + to_string(order.tableNumber) +
                      " | " + order.status + " | " + to_string(order.items.size()) + " блюд " +
                      "(Ожидание: " + to_string(waiting) + ", Готовится: " + to_string(cooking) +
                      ", Готово: " + to_string(ready) + ", Подано: " + to_string(served) + ")";
        SendMessageA(hKitchenOrdersList, LB_ADDSTRING, 0, (LPARAM)item.c_str());
    }
}
```

```
void RefreshKitchenLog() {
   SendMessage(hKitchenLogList, LB_RESETCONTENT, 0, 0);
   auto log = dataManager.getKitchen().getKitchenLog();
   for (const auto& entry : log) {
      SendMessageA(hKitchenLogList, LB_ADDSTRING, 0,
(LPARAM)entry.c_str());
   }
   int count = SendMessage(hKitchenLogList, LB_GETCOUNT, 0, 0);
   if (count > 0) {
      SendMessage(hKitchenLogList, LB_SETCURSEL, count - 1, 0);
   }
}

void RefreshKitchenStats() {
   auto stats = dataManager.getKitchen().getKitchenStatistics();
   string statsText = "Заказов завершено сегодня: " +
to_string((int)stats["orders_completed_today"]) + "\n" +
               "Активных заказов: " + to_string((int)stats["active_orders"]) + "\n"
+
               "Блюд готовится: " + to_string((int)stats["items_cooking"]) + "\n" +
               "Блюд в ожидании: " + to_string((int)stats["items_waiting"]) + "\n"
+
               "Блюд готово: " + to_string((int)stats["items_ready"]) + "\n" +
               "Блюд подано: " + to_string((int)stats["items_served"]) + "\n" +
               "Выручка за день: " + to_string((int)stats["daily_revenue"]) + "
руб.";
   SetWindowTextA(hKitchenStatsText, statsText.c_str());
}

void RefreshRevenue() {
   double revenue = dataManager.getTotalRevenue();
   string revenueText = to_string((int)revenue) + " руб.";
   SetWindowTextA(hRevenueText, revenueText.c_str());
}

void RefreshOrderDetails(int orderId) {
   SendMessage(hOrderList, LB_RESETCONTENT, 0, 0);
   auto order = dataManager.getOrderById(orderId);
   if (order) {
      SendMessageA(hOrderList, LB_ADDSTRING, 0, (LPARAM)"===
ДЕТАЛИ ЗАКАЗА ===");
      SendMessageA(hOrderList, LB_ADDSTRING, 0, (LPARAM)("Статус: " +
order->status).c_str());
```

```
        // Показываем текущее готовящееся блюдо
        if (order->currentCookingIndex >= 0 && order->currentCookingIndex < order->items.size()) {
            string cooking = "Сейчас готовится: " + order->items[order->currentCookingIndex].item.name;
            SendMessageA(hOrderList, LB_ADDSTRING, 0, (LPARAM)cooking.c_str());
        }

        for (int i = 0; i < order->items.size(); i++) {
            const auto& orderItem = order->items[i];
            string prefix = (i == order->currentCookingIndex) ? ">>> " : "";
            string listItem = prefix + orderItem.item.name + " - " +
                              to_string((int)orderItem.item.price) + " руб. [" + orderItem.status + "]";
            SendMessageA(hOrderList, LB_ADDSTRING, 0, (LPARAM)listItem.c_str());
        }
        string total = "ИТОГО: " + to_string((int)order->getTotal()) + " руб.";
        SendMessageA(hOrderList, LB_ADDSTRING, 0, (LPARAM)total.c_str());

        if (order->canBeCompleted()) {
            SendMessageA(hOrderList, LB_ADDSTRING, 0, (LPARAM)"=== ЗАКАЗ МОЖНО ЗАВЕРШИТЬ ===");
        } else {
            SendMessageA(hOrderList, LB_ADDSTRING, 0, (LPARAM)"=== НЕ ВСЕ БЛЮДА ПОДАНЫ ===");
        }
    }
}

void UpdateStatus(const string& message) {
    string status = "Статус: " + message;
    SetWindowTextA(hStatusText, status.c_str());
}

void UpdateKitchenStatus(const string& message) {
    string status = "Кухня: " + message;
    UpdateStatus(status);
}

// ==================== ОБРАБОТЧИК СОБЫТИЙ ====================
```

```
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
   switch (uMsg) {
      case WM_CREATE:
         CreateHallControls(hwnd);
         CreateKitchenControls(hwnd);
         ShowHallInterface();
         break;

      case WM_COMMAND: {
         int cmd = LOWORD(wParam);

         if (cmd == 10) {
            isKitchenMode = true;
            ShowKitchenInterface();
            break;
         }
         else if (cmd == 11) {
            isKitchenMode = false;
            ShowHallInterface();
            break;
         }

         if (!isKitchenMode) {
            if (cmd == 1) {
               int tableIndex = SendMessage(hTableList, LB_GETCURSEL, 0, 0);
               if (tableIndex >= 0) {
                  auto tables = dataManager.getTables();
                  if (tableIndex < tables.size() && !tables[tableIndex].isOccupied) {
                     int orderId = dataManager.createOrder(tables[tableIndex].number);
                     RefreshTableList();
                     RefreshActiveOrdersList();
                     UpdateStatus("Создан новый заказ для стола " + to_string(tables[tableIndex].number));
                  }
               }
            }
            else if (cmd == 2) {
               int orderIndex = SendMessage(hActiveOrdersList, LB_GETCURSEL, 0, 0);
               int menuIndex = SendMessage(hMenuList, LB_GETCURSEL, 0, 0);

               if (orderIndex >= 0 && menuIndex >= 0) {
```

```
                auto orders = dataManager.getActiveOrders();
                auto menu = dataManager.getMenu();

                if (orderIndex < orders.size() && menuIndex < menu.size()) {
                    dataManager.addItemToOrder(orders[orderIndex].id,
menu[menuIndex].id);
                    RefreshActiveOrdersList();
                    RefreshOrderDetails(orders[orderIndex].id);
                    UpdateStatus("Добавлено блюдо в заказ");
                }
            }
        }
        else if (cmd == 3) {
            int orderIndex = SendMessage(hActiveOrdersList, LB_GETCURSEL,
0, 0);
            if (orderIndex >= 0) {
                auto orders = dataManager.getActiveOrders();
                if (orderIndex < orders.size()) {
                    if (dataManager.completeOrder(orders[orderIndex].id)) {
                        RefreshTableList();
                        RefreshActiveOrdersList();
                        RefreshRevenue();
                        SendMessage(hOrderList, LB_RESETCONTENT, 0, 0);
                        UpdateStatus("Заказ завершен");
                    } else {
                        UpdateStatus("Ошибка: не все блюда поданы!");
                    }
                }
            }
        }
        else if (cmd == 4) {
            int orderIndex = SendMessage(hActiveOrdersList, LB_GETCURSEL,
0, 0);
            if (orderIndex >= 0) {
                auto orders = dataManager.getActiveOrders();
                if (orderIndex < orders.size()) {
                    dataManager.sendOrderToKitchen(orders[orderIndex].id);
                    RefreshActiveOrdersList();
                    RefreshKitchenOrdersList();
                    RefreshKitchenLog();
                    UpdateStatus("Заказ передан на кухню");
                }
            }
        }
```

```
            else if (HIWORD(wParam) == LBN_SELCHANGE) {
                if ((HWND)lParam == hActiveOrdersList) {
                    int orderIndex = SendMessage(hActiveOrdersList,
LB_GETCURSEL, 0, 0);
                    if (orderIndex >= 0) {
                        auto orders = dataManager.getActiveOrders();
                        if (orderIndex < orders.size()) {
                            RefreshOrderDetails(orders[orderIndex].id);
                        }
                    }
                }
            }
        } else {
            if (cmd == 5) {
                int orderIndex = SendMessage(hKitchenOrdersList,
LB_GETCURSEL, 0, 0);
                if (orderIndex >= 0) {
                    auto orders = dataManager.getKitchenOrders();
                    if (orderIndex < orders.size()) {
                        Order* order =
dataManager.getOrderById(orders[orderIndex].id);
                        if (order) {
                            if (dataManager.getKitchen().startNextItem(order)) {
                                RefreshKitchenOrdersList();
                                RefreshKitchenLog();
                                RefreshKitchenStats();
                                UpdateKitchenStatus("Начато приготовление следующего
блюда");
                            } else {
                                UpdateKitchenStatus("Нет блюд для приготовления");
                            }
                        }
                    }
                }
            }
            else if (cmd == 6) {
                int orderIndex = SendMessage(hKitchenOrdersList,
LB_GETCURSEL, 0, 0);
                if (orderIndex >= 0) {
                    auto orders = dataManager.getKitchenOrders();
                    if (orderIndex < orders.size()) {
                        Order* order =
dataManager.getOrderById(orders[orderIndex].id);
                        if (order) {
```

```
                    // Завершаем текущее готовящееся блюдо
                    if (order->currentCookingIndex >= 0 &&
                        order->currentCookingIndex < order->items.size()) {
                        dataManager.getKitchen().completeItem(order,
                            order->currentCookingIndex,
                            "Отлично приготовлено!");
                        RefreshKitchenOrdersList();
                        RefreshKitchenLog();
                        RefreshKitchenStats();
                        RefreshActiveOrdersList();
                        UpdateKitchenStatus("Блюдо готово");
                    } else {
                        // Ищем первое блюдо "Готовится"
                        for (int i = 0; i < order->items.size(); i++) {
                            if (order->items[i].status == "Готовится") {
                                dataManager.getKitchen().completeItem(order, i,
                                    "Отлично приготовлено!");
                                RefreshKitchenOrdersList();
                                RefreshKitchenLog();
                                RefreshKitchenStats();
                                RefreshActiveOrdersList();
                                UpdateKitchenStatus("Блюдо готово");
                                break;
                            }
                        }
                    }
                }
            }
        }
        else if (cmd == 7) {
            int orderIndex = SendMessage(hKitchenOrdersList,
LB_GETCURSEL, 0, 0);
            if (orderIndex >= 0) {
                auto orders = dataManager.getKitchenOrders();
                if (orderIndex < orders.size()) {
                    Order* order =
dataManager.getOrderById(orders[orderIndex].id);
                    if (order) {
                        if (dataManager.getKitchen().serveNextItem(order)) {
                            RefreshKitchenOrdersList();
                            RefreshKitchenLog();
                            RefreshKitchenStats();
                            RefreshActiveOrdersList();
```

```
                        RefreshRevenue();
                        UpdateKitchenStatus("Блюдо подано");
                    } else {
                        UpdateKitchenStatus("Нет готовых блюд для подачи");
                    }
                }
            }
        }
    }
    break;
}

case WM_CLOSE:
    DestroyWindow(hwnd);
    break;

case WM_DESTROY:
    PostQuitMessage(0);
    break;

default:
    return DefWindowProc(hwnd, uMsg, wParam, lParam);
}
return 0;
}
```

Приложение Б. Диаграммы UML

Б.1. Диаграмма последовательности

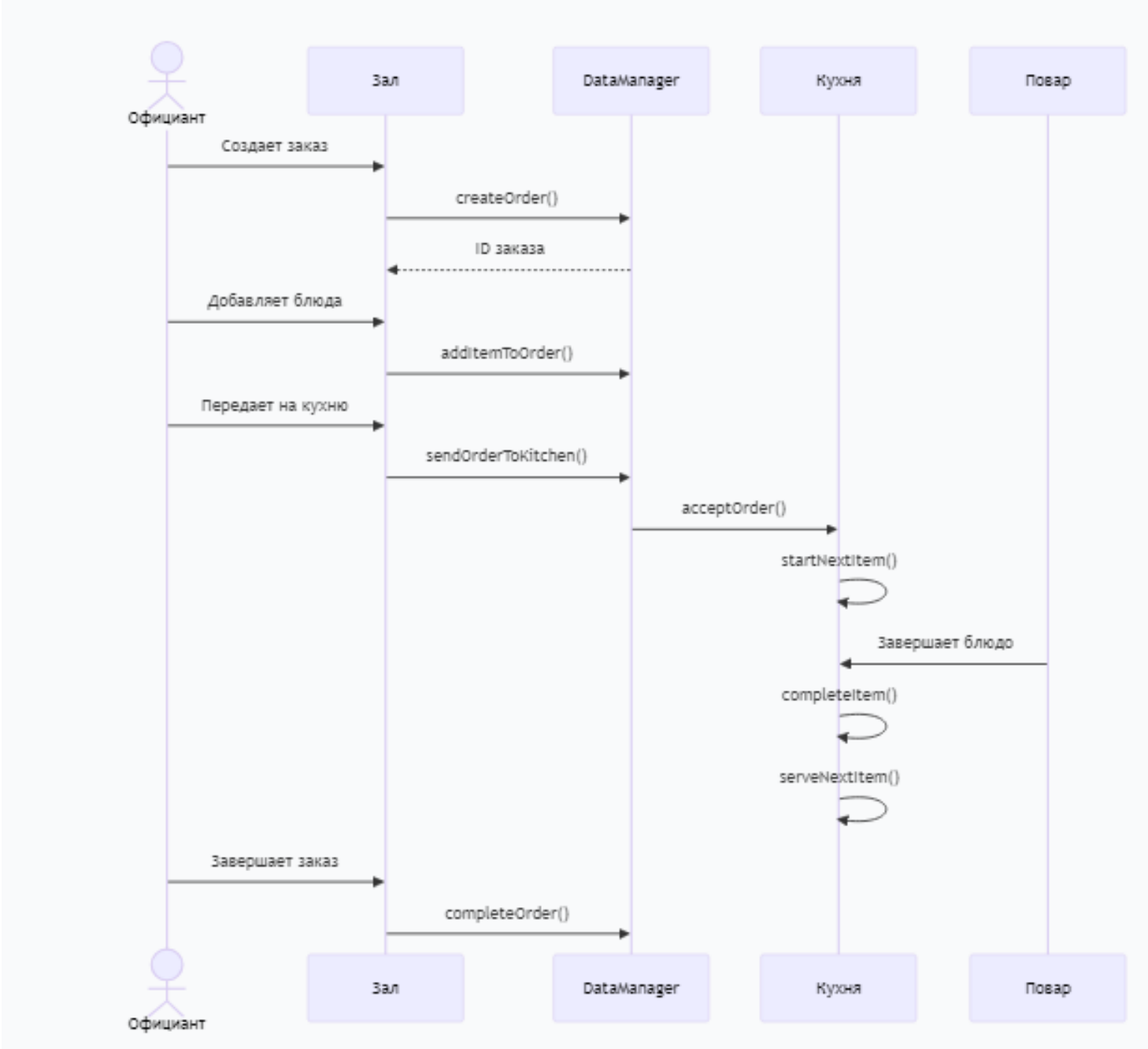


Рисунок 11 - Диаграмма последовательности

Б.2. Диаграмма состояний заказа

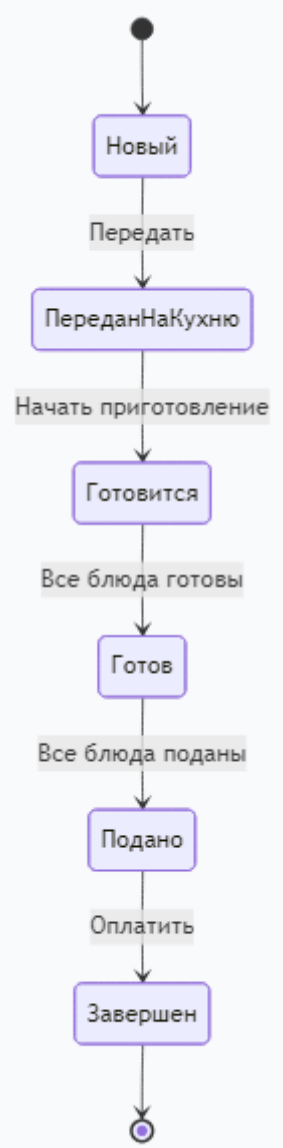


Рисунок 12 - Диаграмма состояний заказа

Приложение В. Руководство по установке

1. **Требования к системе**:
   - Windows 7 или выше
   - 20 MB свободного места
   - Дополнительные библиотеки не требуются (статическая сборка)
2. **Установка**:
   - Распаковать архив с программой
   - Запустить RestaurantOrderSystem.exe
   - При первом запуске программа инициализирует тестовые данные
3. **Обновление**:
   - Закрыть программу
   - Скопировать новые файлы
   - Запустить программу заново
4. **Удаление**:
   - Удалить папку с программой
   - Программа не оставляет данных в системе